

1 Algorithm

1.1 LIS

```
1 // Longest increasing subsequence
2 int LIS(const vector<int>& s) {
3     // use binary search to update the smallest
4     // element
5     // that ends in a subsequence of length d
6     vector<int> v{};
7     v.push_back(s[0]);
8     for (int i{1}; i < s.size(); ++i)
9         if (s[i] > v.back()) v.push_back(s[i]);
10    else *lower_bound(v.begin(), v.end(),
11                     , s[i]) = s[i];
12    return v.size();
13 }
```

1.2 LCS

```
1 // Longest common subsequence
2 int LCS(const vector<int>& a, const vector<
3 int>& b) {
4     vector<vector<int>> dp(a.size() + 1,
5 vector<int>(b.size() + 1));
6     for (int i{1}; i <= int(a.size()); ++i)
7         for (int j{1}; j <= int(b.size());
8             ++j) {
9             dp[i][j] = max(dp[i][j], max(dp[i
10 - 1][j], dp[i][j - 1]));
11             if (a[i - 1] == b[j - 1]) dp[i][j]
12                 = max(dp[i][j], dp[i - 1][j - 1] + 1);
13         }
14     return dp[a.size()][b.size()];
15 }
```

2 Basic

2.1 mod helper function

```
1 int add(int i, int j) {
2     if ((i += j) >= MOD)
3         i -= MOD;
4     return i;
5 }
6
7 int sub(int i, int j) {
8     if ((i -= j) < 0)
9         i += MOD;
10    return i;
11 }
```

2.2 self-defined-pq-operator

```
1 auto cmp = [](int a, int b) {
2     return a > b;
3 };
4 priority_queue<int, vector<int>, decltype(
5 cmp)> pq(cmp);
```

2.3 generating all subsets

```
1 for (int b = 0; b < (1<<n); b++) {
2     vector<int> subset;
3     for (int i = 0; i < n; i++) {
4         if (b & (1<<i)) subset.push_back(vc[i]);
5     }
6 }
```

2.4 tuple

```
1 int main() {
2     tuple<int, double, string> myTuple(100,
3 3.14, "hello world");
4     tuple<int, double, string> anotherTuple
5 = make_tuple(100, 18, "Tom");
6     cout << get<0>(myTuple) << "\n"; // 取得
7     // 第一個元素
8     auto [a, b, c] = myTuple;
9     get<1>(myTuple) = 5.43; // 修改第二個元
10    // 素的值
11
12     int a;
13     double b;
14     string c;
15     tie(a, b, c) = myTuple;
16 }
```

2.5 memset

```
1 memset(a, 0, sizeof(a)); // 0
2 memset(a, 0x3f3f3f3f, sizeof(a)); // INF
```

2.6 submask enumeration

```
1 // O(3^n)
2
3 int m = 0b10110; // binary: 10110, decimal:
4 22
5 cout << "Mask: " << bitset<5>(m) << " (" <<
6 m << ")\n";
```

```
7 // Enumerate all submasks
8 for (int s = m; s; s = (s - 1) & m) {
9     cout << bitset<5>(s) << " (" << s << ")\n";
10 }
11
12 // Optionally include the empty submask
13 cout << bitset<5>(0) << " (0)\n";
```

```
28 ans += mat.mat[i][j];
29 ans %= MOD;
30 }
31 }
32 cout << ans << "\n";
33 }
```

3.2 grouping

```
1 /*
2 狀態 DP - O(3^N * 2^N * N^2)
3 有 N 隻兔子，編號為 1, 2, ..., N。
4
5 對於每一對 i, j (1 ≤ i, j ≤ N)，兔子 i 與 j 的相容
6 度由整數 a_{i,j} 描述。這裡 a_{i,i} = 0 對於
7 每個 i (1 ≤ i ≤ N)，且 a_{i,j} = a_{j,i} 對於任
8 意 i 與 j (1 ≤ i, j ≤ N)。
9
10 A 將 N 隻兔子分成若干個群組。每隻兔子必須且
11 僅屬於一個群組。分群後，對於每一對 i 與
12 j (1 ≤ i < j ≤ N)，若兔子 i 與 j 屬於同一群
13 組，A 即可獲得 a_{i,j} 分。
```

求 A 能獲得的最大總分。

令 $cost[S]$ 表示將集合 S 中的所有兔子放在同一群組時所得到的分數。此值可在 $O(2^N * N^2)$ 時間內計算。

接著我們計算 $dp[S]$ ，表示對集合 S 中的兔子進行分群時所能得到的最大分數。

```
14 /*
15 void solve() {
16     int n;
17     cin >> n;
18     vector<vector<int>> a(n, vector<int>(n));
19     ;
20     vector<int> cost(1<<n, 0);
21     vector<int> dp(1<<n, 0);
22
23     for (int i = 0; i < n; ++i) {
24         for (int j = 0; j < n; ++j) {
25             cin >> a[i][j];
26         }
27     }
28
29     // backtrack all subset
30     for (int b = 0; b < (1<<n); ++b) {
31         vector<int> subset;
32         for (int i = 0; i < n; ++i) {
33             if (b & (1<<i)) {
34                 for (const int& j : subset)
35                     cost[b] += a[i][j];
36             }
37             subset.pb(i);
38         }
39     }
40 }
```

2.7 stringstream split by comma

```
1 while (std::getline(ss, segment, ',')) {
2     segments.push_back(segment);
3 }
```

3 DP

3.1 walk

```
1 /*
2 DP on graphs - O(N^3 Log K)
3 給定一個簡單的有向圖 G，具有 N 個頂點，編號
4 為 1, 2, ..., N。
5
6 對於任意 i, j (1 ≤ i, j ≤ N)，給定整數 a_{i,j}，表示
7 是否存在從頂點 i 指向頂點 j 的有向邊。若 a_{i,j} = 1，則存在邊；若 a_{i,j} = 0，則不存在。
8
9 求圖中長度為 K 的不同有向路徑數目，對 10^9+7
10 取模。路徑可重複通過相同邊（即允許重複
11 邊）。
12
13 注意：當我們將鄰接矩陣 m 與 m 相乘時，得到的是
14 長度為 2 的路徑數；若取 m 的 p 次方 m^p，則其 (i, j)
15 元素表示從 i 到 j 的長度為 p 的路徑數。
```

```
16 void solve() {
17     int n, k;
18     cin >> n >> k;
19     vector<vector<int>> m(n, vector<int>(n));
20     ;
21
22     for (int i = 0; i < n; ++i) {
23         for (int j = 0; j < n; ++j) {
24             cin >> m[i][j];
25         }
26     }
27
28     Matrix<int> mat(m);
29     mat = power(mat, k);
30     int ans = 0;
31     for (int i = 0; i < n; ++i) {
32         for (int j = 0; j < n; ++j) {
```

```

41 // dp
42 for (int i = 0; i < (1<<n); ++i) {
43     int j = ((1<<n) - 1) ^ i;
44     for (int s = j; s != 0; s = (s - 1)
45         & j) {
46         dp[i ^ s] = max(dp[i ^ s], dp[i]
47             + cost[s]);
48     }
49     cout << dp[(1<<n) - 1] << "\n";
50 }

```

3.3 matching

```

1 /*
2 Bitmask DP -  $O(N * 2^N)$ 
3 有  $N$  個男人和  $N$  個女人，分別編號為 1, 2, ...,
4      $N$ 。
5 對於每個  $i, j$  ( $1 \leq i, j \leq N$ )，男人  $i$  和女人
6      $j$  的相容性由整數  $a[i][j]$  給出。
7 如果  $a[i][j] = 1$ ，則男人  $i$  和女人  $j$  是相容
8     的；
9 如果  $a[i][j] = 0$ ，則不是。
10 A 正在嘗試組成  $N$  對，每對由一個相容的男人和
11     女人組成。在這裡，每個男人和每個女人必須
12     恰好屬於一對。
13 求 A 可以組成  $N$  對的方法數，結果對  $10^9 + 7$ 
14     取模。
15 定義  $dp[S]$  為將集合  $S$  中的女性與前  $|S|$  個男
16     性配對的方法數。
17 */
18 const int maxn = 21;
19 const int MOD = 1e9 + 7;
20 int n;
21 int grid[maxn][maxn];
22 int dp[1 << maxn];
23 void solve() {
24     cin >> n;
25     memset(dp, 0, sizeof(dp));
26     for (int i = 0; i < n; ++i) {
27         for (int j = 0; j < n; ++j) {
28             cin >> grid[i][j];
29         }
30     }
31     dp[0] = 1;
32     for (int s = 0; s < (1 << n); ++s) {
33         int ps = __builtin_popcount(s);
34         for (int w = 0; w < n; ++w) {
35             if ((s & (1 << w)) || !grid[ps][
36                 w]) {
37                 continue;
38             }

```

```

39         dp[s | (1 << w)] += dp[s];
40         dp[s | (1 << w)] %= MOD;
41     }
42     cout << dp[(1 << n) - 1] << "\n";
43 }
44 }
45 }

```

3.4 projects

```

1 /*
2 LIS DP -  $O(N \log N)$ 
3 有  $n$  個你可以參加的專案。對於每個專案，你知
4     道其開始與結束天數以及可獲得的報酬金額。
5 在同一天你最多只能參加一個專案。
6 問：你最多可以賺到多少金額？
7  $dp[i]$  = 在第  $i$  天之前我們可以賺到的最大金
8     額。
9 */
10 void solve() {
11     int n;
12     cin >> n;
13     vector<array<int, 3>> vc(n);
14     map<int, int> days;
15     for (int i = 0; i < n; ++i) {
16         int a, b, p;
17         cin >> a >> b >> p;
18         days[a] = days[b] = 1;
19         vc[i] = {a, b, p};
20     }
21     int idx = 1;
22     for (auto& x : days) {
23         x.second = idx++;
24     }
25     vector<int> dp(idx, 0);
26     sort(vc.begin(), vc.end(), [](const
27         array<int, 3>& va, const array<int,
28         3>& vb) {
29         if (va[1] != vb[1]) return va[1] <
30             vb[1];
31         if (va[0] != vb[0]) return va[0] <
32             vb[0];
33         return va[2] > vb[2];
34     });
35     int i = 0;
36     for (int d = 1; d < idx; ++d) {
37         dp[d] = dp[d - 1];
38         while (i < n && days[vc[i][1]] == d)
39             {
40                 dp[d] = max(dp[d], dp[days[vc[i]
41                     ][0]] - 1 + vc[i][2]);
42                 i++;
43             }
44     }
45     cout << dp[idx - 1] << "\n";
46 }

```

3.5 elevator rides

```

1 /*
2 狀態  $DP - O(2^N)$ 
3 有  $n$  個人想要搭電梯到樓頂，建築物只有一部電
4     梯。你知道每個人的體重以及電梯的最大允許
5     載重。最少需要搭乘多少次電梯？
6 定義  $dp[S] = \{r, w\}$ ，其中  $r$  是將集合  $S$  中的
7     所有人送到樓頂所需的最少電梯次數， $w$  是最
8     後一次電梯所載人的總重量。
9 */
10 void solve() {
11     int n, x;
12     cin >> n >> x;
13     vector<int> w(n);
14     vector<pii> dp(1<<n, {INF, INF});
15     for (int i = 0; i < n; ++i) {
16         cin >> w[i];
17     }
18     dp[0] = {1, 0};
19     for (int b = 1; b < (1<<n); ++b) {
20         for (int i = 0; i < n; ++i) {
21             if (b & (1<<i)) {
22                 auto [r_prev, w_prev] = dp[b
23                     ^ (1<<i)];
24                 pii can;
25                 if (w_prev + w[i] <= x) {
26                     can = {r_prev, w_prev +
27                         w[i]};
28                 }
29                 else {
30                     can = {r_prev + 1, w[i]
31                         };
32                 }
33                 dp[b] = min(dp[b], can);
34             }
35         }
36     }
37     cout << dp[(1<<n) - 1].first << "\n";
38 }

```

3.6 digit sum

```

1 /*
2 Digit DP -  $O(|K| * D)$ 
3 計算在 1 到  $K$  (含) 之間，滿足其十進位數字和
4     為  $D$  的倍數的整數數量，答案對  $10^9 + 7$  取
5     模。
6 令  $dp[i][j]$  表示在已確定前  $i$  位數字的情況
7     下，構成長度為  $|K|$  的數字且目前數字和
8      $\text{mod } D$  等於  $j$  的方法數。
9 */
10 const int MOD = 1e9 + 7;
11 int dp[1001][101][2];

```

```

10 void solve() {
11     string K;
12     int D;
13     cin >> K >> D;
14     int len = K.size();
15     memset(dp, 0, sizeof(dp));
16     dp[0][0][1] = 1;
17     for (int i = 1; i <= len; ++i) {
18         int limit = K[i - 1] - '0';
19         for (int s = 0; s < D; ++s) {
20             for (int flag = 0; flag <= 1; ++
21                 flag) {
22                 int ways = dp[i - 1][s][flag
23                     ];
24                 if (ways == 0) continue;
25                 int max_d = (flag ? limit :
26                     9);
27                 for (int d = 0; d <= max_d;
28                     ++d) {
29                     int rs = (s + d) % D;
30                     int rflag = (flag && d
31                         == max_d ? 1 : 0);
32                     dp[i][rs][rflag] += ways
33                         ;
34                     dp[i][rs][rflag] %= MOD;
35                 }
36             }
37         }
38     }
39     int ans = (dp[len][0][0] + dp[len
40         ][0][1]) % MOD;
41     ans = (ans - 1 + MOD) % MOD;
42     cout << ans << "\n";
43 }

```

3.7 sushi

```

1 /*
2 期望值  $DP - O(N^3)$ 
3 有  $N$  盤壽司，編號從 1 到  $N$ 。第  $i$  盤最初有  $a_i$ 
4     ( $1 \leq a_i \leq 3$ ) 塊壽司。
5 太郎不斷擲一顆編號 1 到  $N$  的骰子。如果結果是
6     第  $i$  盤且該盤還有壽司，他就吃掉一塊；否
7     則什麼也不做。
8 請求出吃完所有壽司所需擲骰子的期望次數。
9  $dp[x][y][z]$  代表還有
10      $x$  盤剩 1 塊壽司、
11      $y$  盤剩 2 塊壽司、
12      $z$  盤剩 3 塊壽司時的期望擲骰次數。
13 */
14 const int maxn = 301;
15 double dp[maxn][maxn][maxn];
16 int n;

```

```

19 double dfs(int x, int y, int z) {
20     if (x < 0 || y < 0 || z < 0) return 0;
21     if (x == 0 && y == 0 && z == 0) return 0;
22     if (dp[x][y][z] > 0) return dp[x][y][z];
23     double ans = n + x * dfs(x - 1, y, z)
24         + y * dfs(x + 1, y - 1, z)
25         + z * dfs(x, y + 1, z - 1);
26     return dp[x][y][z] = ans / (x + y + z);
27 }
28
29 void solve() {
30     cin >> n;
31     vector<int> a(n);
32     memset(dp, -1, sizeof(dp));
33     vector<int> freq(4, 0);
34
35     for (int i = 0; i < n; ++i) {
36         cin >> a[i];
37         freq[a[i]]++;
38     }
39
40     cout << fixed << setprecision(10) << dfs
41         (freq[1], freq[2], freq[3]) << "\n";

```

3.8 candies

```

1 /*
2 組合 DP - O(NK)
3 有 N 個小孩，編號為 1,2,...,N。
4
5 他們決定將 K 顆糖果分給自己。對於每個 i (1≤i
6 ≤N)，第 i 個小孩最多可以拿到 ai 顆糖果
7 (包含 0 顆)。所有糖果都必須分完，不能
8 剩下。
9
10 請問有多少種分配糖果的方法？請將答案對
11 10^9+7 取模。若存在某個小孩分到的糖果數
12 不同，則視為不同的分配方式。
13
14 令 dp[i][j] 表示將 j 顆糖果分給前 i 個小孩的
15 方法數。
16
17 */
18
19 void solve() {
20     int n, k;
21     cin >> n >> k;
22     vector<int> a(n);
23     vector<int> dp(k + 1, 0), S(k + 1, 0);
24
25     for (int i = 0; i < n; ++i) {
26         cin >> a[i];
27     }
28
29     dp[0] = 1;
30     for (int i = 0; i < n; ++i) {
31         vector<int> new_dp(k + 1, 0);
32         S[0] = dp[0];

```

```

26     for (int j = 1; j <= k; ++j) {
27         S[j] = (S[j - 1] + dp[j]) % MOD;
28     }
29     for (int j = 0; j <= k; ++j) {
30         if (j - a[i] - 1 >= 0) {
31             new_dp[j] = (S[j] - S[j - a[i]
32                 - 1] + MOD) % MOD;
33         }
34         else {
35             new_dp[j] = S[j] % MOD;
36         }
37     }
38     dp = new_dp;
39     cout << dp[k] << "\n";
40 }

```

3.9 permutation

```

1 /*
2 抽象 DP - O(N^2)
3 設 N 為正整數。給定一個長度為 N-1 的字串 s，
4 字元僅包含 '<' 與 '>'。
5
6 求滿足條件的排列 (p1, p2, ..., pN) (即 1 到 N
7 的排列) 數量，答案對 10^9+7 取模：
8
9 對於每個 i (1 ≤ i ≤ N-1)，若 s 的第 i 個字
10 元為 '<'，則要求 p_i < p_{i+1}；若為 '>'，
11 則要求 p_i > p_{i+1}。
12
13 令 dp[i][j] 表示：在考慮前 i 個比較符號 (即
14 構成長度為 i+1 的排列) 且最後一個元素為
15 j 的有效排列數量。
16
17 */
18
19 void solve() {
20     int n;
21     string s;
22     cin >> n >> s;
23     vector<vector<int>> dp(n + 1, vector<int>
24         (n + 1, 0));
25     vector<int> prefix(n + 1, 0);
26     dp[1][0] = 1;
27
28     for (int i = 2; i <= n; ++i) {
29         for (int k = 0; k < n; ++k) {
30             prefix[k + 1] = prefix[k] + dp[i
31                 - 1][k];
32         }
33         for (int j = 0; j < i; ++j) {
34             if (s[i - 2] == '>') {
35                 dp[i][j] += prefix[i - 1] -
36                     prefix[j];
37                 dp[i][j] %= MOD;
38             }
39             /*
40             for (int k = j; k < i - 1;
41                 ++k) {
42                 dp[i][j] += dp[i - 1][k]
43                     ];
44             }

```

```

32     */
33     }
34     else {
35         dp[i][j] += prefix[j];
36         dp[i][j] %= MOD;
37     }
38     /*
39     for (int k = 0; k < j; ++k)
40         {
41             dp[i][j] += dp[i - 1][k]
42                 ];
43         }
44     */
45     }
46     }
47     int ans = 0;
48     for (int j = 0; j < n; ++j) {
49         ans += dp[n][j];
50         ans %= MOD;
51     }
52     cout << ans << "\n";

```

3.10 independent set

```

1 /*
2 DP on Trees - O(N)
3 有一棵含 N 個頂點的樹，頂點編號為 1,2,...,N。
4 對於每個 i (1 ≤ i ≤ N-1)，第 i 條邊連接
5 頂點 x_i 和 y_i。
6
7 A 決定將每個頂點塗成白色或黑色，但不允許兩個
8 相鄰的頂點同時為黑色。
9
10 求將頂點塗色的方案數，對 10^9+7 取模。
11
12 設 dp[i][j] 表示以節點 i 為根的子樹中，在節
13 點 i 顏色為 j 時的塗色方案數 (例如 j=0
14 表示白色，j=1 表示黑色)。
15
16 */
17
18 const int maxn = 100001;
19 vector<int> adj[maxn];
20 int f[maxn][2];
21 const int MOD = 1e9 + 7;
22
23 void dp(int u, int p) {
24     for (int v : adj[u]) {
25         if (v != p) {
26             dp(v, u);
27             f[u][0] = (f[v][0] + f[v][1]) %
28                 MOD * f[u][0] % MOD;
29             f[u][1] = f[v][0] * f[u][1] %
30                 MOD;
31         }
32     }
33 }
34
35 void solve() {
36     int n;

```

```

30     cin >> n;
31
32     for (int i = 0; i < n; ++i) {
33         f[i][0] = f[i][1] = 1;
34     }
35
36     for (int i = 0; i < n - 1; ++i) {
37         int u, v;
38         cin >> u >> v;
39         u--, v--;
40         adj[u].pb(v);
41         adj[v].pb(u);
42     }
43
44     dp(0, -1);
45
46     cout << (f[0][0] + f[0][1]) % MOD << "\n";
47 }

```

3.11 counting tower

```

1 /*
2 狀態機 DP - O(N)
3 你的任務是建造一座寬度為 2、高度為 n 的塔。
4 你有無限數量寬度與高度為整數的方塊。
5
6 dp[i][0] = 高度為 i 的塔中，頂層為一個寬度為
7 2 的方塊 (即該層由跨越兩欄的單一方塊覆
8 蓋) 的塔的數量。
9
10 dp[i][1] = 高度為 i 的塔中，頂層在該層有兩個
11 寬度為 1 的方塊 (每欄各一個) 的塔的數
12 量。
13
14 */
15
16 void solve() {
17     long long n;
18     cin >> n;
19     dp[1][0] = 1;
20     dp[1][1] = 1;
21     for (int i = 2; i <= n; ++i) {
22         dp[i][0] = (4 * dp[i-1][0] + dp[i-1][1]) % mod;
23         dp[i][1] = (dp[i-1][0] + 2 * dp[i-1][1]) % mod;
24     }
25     cout << (dp[n][0] + dp[n][1]) % mod << endl;
26 }

```

3.12 counting numbers

```

1 /*
2 區間 DP - O(digits*10)
3 您的任務是計算在區間 a 到 b 之間，沒有任何相
4 鄰兩位數字相同的整數的個數。

```

```

5 定義 dp[pos][prev_digit][is_tight][
    is_started] 為從位置 pos 到結尾的有效整
    數數量。
6 其中 prev_digit 是位置 pos-1 的數字。
7 is_tight 表示是否受原數字前綴的限制。
8 is_started 表示是否已開始構成有效數字（用以
    避免計入前導零）。
9
10 */
11 // dp[pos][prev_digit][is_tight][is_started]
12 int dp[20][10][2][2];
13 string s;
14
15 int f(int pos, int prev_digit, bool is_tight
    , bool is_started) {
16     if (pos == (int)s.size()) {
17         return 1;
18     }
19     if (dp[pos][prev_digit][is_tight][
        is_started] != -1) {
20         return dp[pos][prev_digit][is_tight]
            [is_started];
21     }
22
23     int ans = 0;
24     int max_d = is_tight ? (s[pos] - '0') :
        9;
25     for (int d = 0; d <= max_d; ++d) {
26         if (is_started && d == prev_digit) {
27             continue;
28         }
29
30         bool new_is_started = is_started ||
            (d > 0);
31         bool new_is_tight = is_tight && (d
            == max_d);
32         ans += f(pos + 1, d, new_is_tight,
            new_is_started);
33     }
34
35     return dp[pos][prev_digit][is_tight][
        is_started] = ans;
36 }
37
38 int count(int x) {
39     if (x < 0) return 0;
40     s = to_string(x);
41     memset(dp, -1, sizeof(dp));
42     return f(0, 0, true, false);
43 }
44
45 void solve() {
46     int a, b;
47     cin >> a >> b;
48
49     int ca = count(a - 1);
50     int cb = count(b);
51     cout << cb - ca << "\n";
52 }

```

4 Data Structure

4.1 undo disjoint set

```

1 struct DisjointSet {
2     // save() is like recursive
3     // undo() is like return
4     int n, fa[MXN], sz[MXN];
5     vector<pair<int*,int>> h;
6     vector<int> sp;
7     void init(int tn) {
8         n=tn;
9         for (int i=0; i<n; i++) sz[fa[i]=i]=1;
10        sp.clear(); h.clear();
11    }
12    void assign(int *k, int v) {
13        h.PB({k, *k});
14        *k=v;
15    }
16    void save() { sp.PB(SZ(h)); }
17    void undo() {
18        assert(!sp.empty());
19        int last=sp.back(); sp.pop_back();
20        while (SZ(h)!=last) {
21            auto x=h.back(); h.pop_back();
22            *x.F=x.S;
23        }
24    }
25    int f(int x) {
26        while (fa[x]!=x) x=fa[x];
27        return x;
28    }
29    void uni(int x, int y) {
30        x=f(x); y=f(y);
31        if (x==y) return;
32        if (sz[x]<sz[y]) swap(x, y);
33        assign(&sz[x], sz[x]+sz[y]);
34        assign(&fa[y], x);
35    }
36 } djs;

```

4.2 lazy propagation

```

1 // Lazy Propagation Segment Tree supporting
    range add, range set, and range sum
    queries
2
3 int n;
4 int sz;
5 struct Node {
6     int sum = 0;
7     int lazy_add = 0;
8     int lazy_set = 0;
9     bool has_set = false;
10 };
11 vector<Node> tree;
12
13 void build(const vector<int>& a) {
14     tree.resize(sz << 1);
15     for (int i = 0; i < sz; ++i) {

```

```

16         tree[i + sz].sum = i < n ? a[i] : 0;
17     }
18     for (int i = sz - 1; i >= 1; --i) {
19         tree[i].sum = tree[i << 1].sum +
            tree[i << 1 | 1].sum;
20     }
21 }
22
23 void apply_set(int idx, int l, int r, int
    val) {
24     tree[idx].sum = (r - l + 1) * val;
25     tree[idx].lazy_set = val;
26     tree[idx].lazy_add = 0;
27     tree[idx].has_set = true;
28 }
29
30 void apply_add(int idx, int l, int r, int
    val) {
31     if (tree[idx].has_set) {
32         tree[idx].lazy_set += val;
33     }
34     else {
35         tree[idx].lazy_add += val;
36     }
37     tree[idx].sum += (r - l + 1) * val;
38 }
39
40 void push(int idx, int l, int r) {
41     if (l == r) return;
42
43     int mid = l + (r - l) / 2;
44     int left = idx << 1, right = idx << 1 |
        1;
45
46     if (tree[idx].has_set) {
47         // propagate the set operation
48         apply_set(left, l, mid, tree[idx].
            lazy_set);
49         apply_set(right, mid + 1, r, tree[
            idx].lazy_set);
50         tree[idx].has_set = false;
51         tree[idx].lazy_set = 0;
52         tree[idx].lazy_add = 0;
53     }
54     else if (tree[idx].lazy_add != 0) {
55         // propagate the add operation
56         apply_add(left, l, mid, tree[idx].
            lazy_add);
57         apply_add(right, mid + 1, r, tree[
            idx].lazy_add);
58         tree[idx].lazy_add = 0;
59     }
60 }
61
62 void range_set(int idx, int l, int r, int ql
    , int qr, int val) {
63     if (qr < l || ql > r) return;
64     if (ql <= l && r <= qr) {
65         apply_set(idx, l, r, val);
66         return;
67     }
68
69     push(idx, l, r);
70     int mid = (l + r) / 2;
71     range_set(idx << 1, l, mid, ql, qr, val)
        ;

```

```

72     range_set(idx << 1 | 1, mid + 1, r, ql,
    qr, val);
73     tree[idx].sum = tree[idx << 1].sum +
        tree[idx << 1 | 1].sum;
74 }
75
76 void range_add(int idx, int l, int r, int ql
    , int qr, int val) {
77     if (ql > r || qr < l) return;
78     if (ql <= l && r <= qr) {
79         apply_add(idx, l, r, val);
80         return;
81     }
82
83     push(idx, l, r);
84     int mid = (l + r) / 2;
85     range_add(idx << 1, l, mid, ql, qr, val)
        ;
86     range_add(idx << 1 | 1, mid + 1, r, ql,
    qr, val);
87     tree[idx].sum = tree[idx << 1].sum +
        tree[idx << 1 | 1].sum;
88 }
89
90 int range_sum(int idx, int l, int r, int ql,
    int qr) {
91     if (qr < l || ql > r) return 0;
92     if (ql <= l && r <= qr) return tree[idx]
        .sum;
93
94     push(idx, l, r);
95     int mid = (l + r) / 2;
96     return range_sum(idx << 1, l, mid, ql,
        qr) +
97         range_sum(idx << 1 | 1, mid + 1,
        r, ql, qr);
98 }
99
100 void solve() {
101     int q;
102     cin >> n >> q;
103     vector<int> a(n);
104
105     sz = 1;
106     while (sz < n) sz <= 1;
107     for (int i = 0; i < n; ++i) {
108         cin >> a[i];
109     }
110
111     build(a);
112
113     while (q--) {
114         int mode, l, r, x;
115         cin >> mode >> l >> r;
116         l--, r--;
117         if (mode == 1) {
118             cin >> x;
119             range_add(1, 0, sz - 1, l, r, x)
                ;
120         }
121         else if (mode == 2) {
122             cin >> x;
123             range_set(1, 0, sz - 1, l, r, x)
                ;
124         }
125         else {

```

```

126         cout << range_sum(1, 0, sz - 1,
127             1, r) << "\n";
128     }
129 }

```

4.3 Lazy heap

```

1 template<typename T, typename Compare = less
2 <T>>
3 struct lp {
4     priority_queue<T, vector<T>, Compare> pq;
5     unordered_map<T, int> cnt;
6     size_t sz = 0;
7     void real_remove() {
8         while(!pq.empty() && cnt[pq.top()] >
9             0) {
10             cnt[pq.top()]--;
11             pq.pop();
12         }
13     }
14     size_t size() {
15         return sz;
16     }
17     void remove(T n) {
18         cnt[n]++;
19         sz--;
20     }
21     T top() {
22         real_remove();
23         return pq.top();
24     }
25     void push(T n) {
26         pq.push(n);
27         sz++;
28     }
29     void pop() {
30         real_remove();
31         pq.pop();
32     }
33 };

```

4.4 Fenwick tree (BIT)

```

1 struct Fenwick {
2     int n;
3     vector<int> bit, prefix;
4     Fenwick(int _n) : n(_n), bit(n + 1, 0),
5         prefix(n + 1, 0) {}
6     void update(int i, int delta) {
7         for (; i <= n; i += i & -i) {
8             bit[i] += delta;
9         }
10    }
11    int query(int i) {
12        int ans = 0;
13        for (; i >= 1; i -= i & -i) {
14            ans += bit[i];
15        }
16    }
17 };

```

```

14     }
15     return ans;
16 }
17 int range(int l, int r) {
18     return query(r) - query(l - 1);
19 }
20 };
21
22 int main() {
23     int n = 5;
24     vector<int> a = {0, 1, 2, 3, 4, 5}; //
25     // 1-indexed
26     Fenwick ft(n);
27     for (int i = 1; i <= n; ++i) ft.update(i, a[i]);
28
29     cout << ft.query(3) << "\n"; // 6
30     cout << ft.range(2, 4) << "\n"; // 9
31
32     ft.update(3, 10); //
33     // a[3] += 10
34     cout << ft.query(3) << "\n"; // 16
35     cout << ft.range(1, 5) << "\n"; // 25
36 }

```

4.5 BIT - range update + range prefix sum

```

1 int n, q;
2
3 void solve() {
4     cin >> n >> q;
5     Fenwick bit_values(n), bit_count(n);
6
7     while (q--) {
8         int mode, l, r, val;
9         cin >> mode;
10        if (mode == 0) {
11            cin >> l >> r >> val;
12            bit_values.update(l, val);
13            bit_count.update(l, val * (1 - 1));
14            bit_values.update(r + 1, -val);
15            bit_count.update(r + 1, -val * r);
16        }
17        else {
18            cin >> l >> r;
19            int pref_l = bit_values.query(l - 1) * (l - 1) - bit_count.query(l - 1);
20            int pref_r = bit_values.query(r) * r - bit_count.query(r);
21            cout << pref_r - pref_l << "\n";
22        }
23    }
24 }

```

4.6 Order Statistics Tree

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4
5 template <class T>
6 using ordered_set =
7     tree<T, null_type, less<T>, rb_tree_tag,
8         tree_order_statistics_node_update>;
9
10 int main() {
11     int n;
12     cin >> n;
13     ordered_set<int> st;
14     vector<int> p(n);
15     for (int i = 0; i < n; ++i) {
16         st.insert(i);
17         cin >> p[i];
18     }
19     for (int i = 0; i < n; ++i) {
20         int ind;
21         cin >> ind;
22         ind--;
23         int pos = *st.find_by_order(ind);
24         cout << st.order_of_key(pos) << ' ';
25         st.erase(pos);
26         cout << p[pos] << (i == n - 1 ? '\n' : ' ');
27     }
28 }

```

4.7 Trie (Prefix tree)

```

1 struct trie {
2     int n = 0;
3     trie *a[2];
4     trie() {
5         a[0] = a[1] = nullptr;
6     }
7
8     void insert(int k) {
9         trie *curr = this;
10        for (int i = 63; i >= 0; --i) {
11            bool bit = (k & (1LL << i)) > 0;
12            if (curr->a[bit] == nullptr) {
13                curr->a[bit] = new trie();
14            }
15            curr = curr->a[bit];
16            curr->n++;
17        }
18    }
19
20    void erase(int k) {
21        trie *curr = this;
22        for (int i = 63; i >= 0; --i) {
23            bool bit = (k & (1LL << i)) > 0;
24            curr = curr->a[bit];
25            curr->n--;
26        }
27    }
28
29    int query(int k) {
30        trie *curr = this;
31        int x = 0;
32    }
33 }

```

```

32     for (int i = 63; i >= 0; --i) {
33         x ^= (1LL << i) & k;
34         x ^= 1LL << i;
35         bool bit = (k & (1LL << i)) == 0;
36         if (curr->a[bit] == nullptr ||
37             curr->a[bit]->n == 0) {
38             x ^= 1LL << i, bit ^= 1;
39             curr = curr->a[bit];
40         }
41         return x;
42     }
43 };

```

4.8 segment tree

```

1 int sz = 1;
2 const int maxn = 2e6;
3 int t[maxn << 1];
4
5 void build(const vector<int>& a) {
6     for (int i = 0; i < n; ++i) {
7         t[i + sz] = a[i];
8     }
9     for (int i = sz - 1; i >= 1; --i) {
10        t[i] = max(t[i << 1], t[i << 1 | 1]);
11    }
12 }
13
14 void update(int i, int val) {
15     i += sz;
16     t[i] = val;
17     for (i >= 1; i >= 1; i >= 1) {
18         t[i] = max(t[i << 1], t[i << 1 | 1]);
19     }
20 }
21
22 int query(int r) {
23     if (t[1] < r) return 0;
24
25     int i = 1;
26     while (i < sz) {
27         if (t[i << 1] >= r) i = i << 1;
28         else i = i << 1 | 1;
29     }
30     return i - sz + 1;
31 }

```

4.9 BIT range update point query

```

1 // Fenwick Tree (Binary Indexed Tree) for
2 // Range Updates and Point Queries
3
4 template<typename T>
5 class BIT {
6     #define ALL(x) begin(x), end(x)
7     private:
8     vector<T> arr;
9 };

```



```

8 int n;
9 inline int lowbit(int x) { return x & (-
10 x); }
11 void addInternal(int s, T v) {
12     while (s > 0) {
13         arr[s] += v;
14         s -= lowbit(s);
15     }
16 public:
17     void init(int n_) {
18         n = n_;
19         arr.resize(n + 1);
20         fill(ALL(arr), 0);
21     }
22     void add(int l, int r, T v) {
23         // add v to interval (l, r], 1-based
24         addInternal(l, -v);
25         addInternal(r, v);
26     }
27     T query(int x) {
28         // value at index x
29         T res = 0;
30         while (x <= n) {
31             res += arr[x];
32             x += lowbit(x);
33         }
34         return res;
35     }
36 #undef ALL
37 };
38
39 int main() {
40     BIT<int> bit;
41     bit.init(5);
42
43     // add +3 to indices 1..3
44     bit.add(0, 3, 3);
45
46     // add +2 to indices 3..5
47     bit.add(2, 5, 2);
48
49     cout << "Value at 1 = " << bit.query(1)
50     << "\n"; // expect 3
51     cout << "Value at 3 = " << bit.query(3)
52     << "\n"; // expect 3+2=5
53     cout << "Value at 5 = " << bit.query(5)
54     << "\n"; // expect 2
55 }

```

4.10 DSU

```

1 // fast disjoint set union
2 class DSU {
3     vector<int> pa, sz;
4 public:
5     explicit DSU(int n) : pa(n, -1), sz(n,
6     1) {}
7     int find(int x) { // collapsing find
8         return pa[x] == -1 ? x : pa[x] =
9         find(pa[x]);
10    }

```

```

9 void unite(int x, int y) { // weighted
10     union
11     auto rx{find(x)}, ry{find(y)};
12     if (rx == ry) return;
13     if (sz[rx] < sz[ry]) swap(rx, ry);
14     pa[ry] = rx, sz[rx] += sz[ry];
15 };

```

5 Geometry

5.1 cross. product

```

1 // If cross(a,b,c) > 0, c is to the left of
2 // line ab
3 // If cross(a,b,c) < 0, c is to the right of
4 // line ab
5 // If cross(a,b,c) = 0, a, b, c are
6 // collinear
7
8 // point struct version
9 // 2D Point structure
10 struct Point {
11     double x, y;
12 };
13
14 // Cross product (scalar in 2D)
15 double cross(const Point& a, const Point& b,
16 const Point& c) {
17     // Computes (b - a) x (c - a)
18     return (b.x - a.x) * (c.y - a.y) -
19     (b.y - a.y) * (c.x - a.x);
20 }
21
22 // std::complex version
23 typedef std::complex<double> point;
24 #define x real()
25 #define y imag()
26
27 double cross(const point &a, const point &b)
28 {
29     return (std::conj(a) * b).imag();
30 }

```

5.2 Check Point in Polygon

```

1 /*
2 We can cast a ray from the point P going in
3 any fixed direction (people usually go
4 to the right).
5 If the point is located on the outside of
6 the polygon the ray will intersect its
7 edges an even number of times.
8 If the point is on the inside of the polygon
9 then it will intersect the edge an odd
10 number of times.
11 */

```

```

7 struct Point {
8     int x, y;
9 };
10
11 int cross(const Point& a, const Point& b,
12 const Point& c) {
13     // return (c - a) x (c - b)
14     // return (c.x - a.x) * (c.y - b.y) - (c
15     .x - b.x) * (c.y - a.y);
16     if (ans == 0) return 0;
17     return ans > 0 ? 1 : -1;
18 }
19
20 bool intersect(const Point& a, const Point&
21 b, const Point& c, const Point& d) {
22     int c1 = cross(a, b, c);
23     int c2 = cross(a, b, d);
24     int c3 = cross(c, d, a);
25     int c4 = cross(c, d, b);
26
27     return (c1 * c2 < 0 && c3 * c4 < 0);
28 }
29
30 bool onSegment(const Point& a, const Point&
31 b, const Point& c) {
32     return (cross(a, b, c) == 0 &&
33     min(a.x, b.x) <= c.x && c.x <=
34     max(a.x, b.x) &&
35     min(a.y, b.y) <= c.y && c.y <=
36     max(a.y, b.y));
37 }
38
39 void solve() {
40     int n, m;
41     cin >> n >> m;
42     vector<Point> poly(n);
43
44     for (int i = 0; i < n; ++i) {
45         int a, b;
46         cin >> a >> b;
47         poly[i] = {a, b};
48     }
49
50     // set a point at outside point to form
51     // the ray
52     Point outside = {(int)2e9 + 7, (int)2e9
53     + 13};
54     for (int i = 0; i < m; ++i) {
55         int a, b;
56         bool found = false;
57         cin >> a >> b;
58         Point p = {a, b};
59
60         int cnt = 0;
61         if (intersect(poly.back(), poly[0],
62 p, outside)) {
63             cnt++;
64         }
65         if (onSegment(poly.back(), poly[0],
66 p)) {
67             cout << "BOUNDARY\n";
68             found = true;
69         }
70         for (int j = 0; j < n - 1 && !found;
71 ++j) {

```

```

61         if (intersect(poly[j], poly[j +
62 1], p, outside)) {
63             cnt++;
64         }
65         if (onSegment(poly[j], poly[j +
66 1], p)) {
67             cout << "BOUNDARY\n";
68             found = true;
69         }
70         if (found) continue;
71         if (cnt & 1) {
72             cout << "INSIDE\n";
73         }
74         else {
75             cout << "OUTSIDE\n";
76         }
77     }

```

5.3 Point

```

1 #pragma once
2
3 template <class T> int sgn(T x) { return (x
4 > 0) - (x < 0); }
5
6 template <class T>
7 struct Point {
8     typedef Point P;
9     T x, y;
10     explicit Point(T x=0, T y=0) : x(x), y(y)
11     {}
12     bool operator<(P p) const { return tie(x,y)
13     < tie(p.x,p.y); }
14     bool operator==(P p) const { return tie(x,
15 y)==tie(p.x,p.y); }
16     P operator+(P p) const { return P(x+p.x, y
17 +p.y); }
18     P operator-(P p) const { return P(x-p.x, y
19 -p.y); }
20     P operator*(T d) const { return P(x*d, y*d)
21     };
22     P operator/(T d) const { return P(x/d, y/d)
23     };
24     T dot(P p) const { return x*p.x + y*p.y; }
25     T cross(P p) const { return x*p.y - y*p.x; }
26     T cross(P a, P b) const { return (a-*this)
27     .cross(b-*this); }
28     T dist2() const { return x*x + y*y; }
29     double dist() const { return sqrt((double)
30 dist2()); }
31     // angle to x-axis in interval [-pi, pi]
32     double angle() const { return atan2(y, x); }
33 }
34
35 P unit() const { return *this/dist(); } //
36 makes dist()=1
37 P perp() const { return P(-y, x); } //
38 rotates +90 degrees
39 P normal() const { return perp().unit(); }
40 // returns point rotated 'a' radians ccw
41 around the origin
42 P rotate(double a) const {

```

```

27     return P(x*cos(a)-y*sin(a),x*sin(a)+y*
        cos(a)); }
28 friend ostream& operator<<(ostream& os, P
    p) {
29     return os << "(" << p.x << ", " << p.y <<
        ")"; }
30 };

```

5.4 Polygon Lattice Points

```

1  /*
2  Pick's Theorem:
3  For a simple polygon with integer
    coordinates, the area A, the number of
    interior lattice points a
4  and the number of boundary lattice points b
    satisfy:
5  A = a + b/2 - 1
6  */
7
8  /*
9  The number of intersections of a line with
    lattice points is the greatest common
    divisor of
10 P1.x - P2.x and P1.y - P2.y
11 */
12 int countOnSegment(const Point& a, const
    Point& b) {
13     int dx = abs(a.x - b.x);
14     int dy = abs(a.y - b.y);
15     return __gcd(dx, dy);
16 }
17
18 void solve() {
19     int n;
20     cin >> n;
21     vector<Point> poly(n);
22
23     for (int i = 0; i < n; ++i) {
24         cin >> poly[i].x >> poly[i].y;
25     }
26
27     // b: boundary points
28     int b = countOnSegment(poly.back(), poly
        [0]);
29     for (int i = 0; i < n - 1; ++i) {
30         b += countOnSegment(poly[i], poly[i
            + 1]);
31     }
32     int area2 = abs(polygonArea2(poly));
33     // a: interior points
34     int a = (area2 + 2 - b) / 2;
35     cout << a << " " << b << "\n";
36 }

```

5.5 line intersect

```

1  // 2D Point structure
2  struct Point {
3      double x, y;

```

```

4  };
5
6  // Cross product (scalar in 2D)
7  double cross(const Point& a, const Point& b,
    const Point& c) {
8      // Computes (b - a) x (c - a)
9      return (b.x - a.x) * (c.y - a.y) -
        (b.y - a.y) * (c.x - a.x);
10 }
11
12 // Check if value is between two others (
    with endpoints allowed)
13 bool between(double a, double b, double x) {
14     return min(a, b) <= x && x <= max(a, b);
15 }
16
17 // Check if point c lies on segment ab
18 bool onSegment(const Point& a, const Point&
    b, const Point& c) {
19     return cross(a, b, c) == 0 &&
        between(a.x, b.x, c.x) &&
        between(a.y, b.y, c.y);
20 }
21
22 // Main intersection check
23 bool segmentsIntersect(const Point& A, const
    Point& B, const Point& C, const
    Point& D) {
24
25     double c1 = cross(A, B, C);
26     double c2 = cross(A, B, D);
27     double c3 = cross(C, D, A);
28     double c4 = cross(C, D, B);
29
30     // General case
31     if (c1 * c2 < 0 && c3 * c4 < 0) return
        true;
32
33     // Special cases: collinear +
        overlapping
34     if (c1 == 0 && onSegment(A, B, C))
35         return true;
36     if (c2 == 0 && onSegment(A, B, D))
37         return true;
38     if (c3 == 0 && onSegment(C, D, A))
39         return true;
40     if (c4 == 0 && onSegment(C, D, B))
41         return true;
42
43     return false;
44 }
45
46 int main() {
47     Point A{0, 0}, B{4, 4};
48     Point C{0, 4}, D{4, 0};
49
50     if (segmentsIntersect(A, B, C, D))
51         cout << "Segments intersect\n";
52     else
53         cout << "Segments do not intersect\n";
54
55     return 0;
56 }

```

5.6 Polygon area

```

1  // Returns twice the signed area of a
    polygon.
2  #pragma once
3
4  #include "Point.h"
5
6  template<class T>
7  T polygonArea2(vector<Point<T>>& v) {
8      T a = v.back().cross(v[0]);
9      for (int i = 0; i < (int)v.size() - 1;
        ++i) {
10         a += v[i].cross(v[i+1]);
11     }
12     return a;
13 }
14
15 int main() {
16     // Example: square with vertices (0,0),
        (0,1), (1,1), (1,0)
17     vector<Point<long long>> poly = {
18         {0,0}, {0,1}, {1,1}, {1,0}
19     };
20
21     long long area2 = polygonArea2(poly);
22     cout << "Twice signed area = " << area2
        << "\n";
23     cout << "Absolute area = " << abs(area2)
        / 2.0 << "\n";
24
25     return 0;
26 }

```

5.7 using std::complex

```

1  #include <iostream>
2  #include <complex>
3  #include <cmath>
4
5  typedef std::complex<double> point;
6  #define x real()
7  #define y imag()
8
9  constexpr double PI = std::acos(-1.0);
10 // conj(a) = a 的共軛 · reflection of a
    across x-axis
11
12 // Basic operations
13 double dot(const point &a, const point &b) {
14     return (std::conj(a) * b).real(); }
15 double cross(const point &a, const point &b) {
16     return (std::conj(a) * b).imag(); }
17
18 // Projections / reflections / geometry
    helpers
19 point project_onto_vector(const point &p,
    const point &v) {
20     return v * (dot(p, v) / std::norm(v));
21 }

```

```

21 point project_onto_line(const point &p,
    const point &a, const point &b) {
22     return a + (b - a) * (dot(p - a, b - a)
        / std::norm(b - a));
23 }
24
25 point reflect_across_line(const point &p,
    const point &a, const point &b) {
26     return a + std::conj((p - a) / (b - a))
        * (b - a);
27 }
28
29 point intersection(const point &a, const
    point &b, const point &p, const point &q) {
30     double c1 = cross(p - a, b - a), c2 =
        cross(q - a, b - a);
31     return (c1 * q - c2 * p) / (c1 - c2); //
        undefined if parallel (divide by
        zero)
32 }
33
34 double angle_between(const point &a, const
    point &b) {
35     // angle of vector b - a
36     return std::arg(b - a);
37 }
38
39 double angle_ABC(const point &a, const point
    &b, const point &c) {
40     double r = std::remainder(std::arg(a - b)
        - std::arg(c - b), 2.0 * PI);
41     return std::abs(r);
42 }
43
44 int main() {
45     // sample
46     point a = 2.0; // (2,0)
47     point b(3.0, 7.0); // (3,7)
48     std::cout << a << ' ' << b << '\n'; //
        (2,0) (3,7)
49     std::cout << a + b << '\n'; //
        (5,7)
50
51     // usage examples
52     point p1(3, 2), p2(2, -7);
53     std::cout << "p1 + p2 = " << p1 + p2 <<
        '\n'; // (5,-5)
54     std::cout << "p1 - p2 = " << p1 - p2 <<
        '\n'; // (1,9)
55     std::cout << "3.0 * p1 = " << 3.0 * p1
        << '\n'; // (9,6)
56     std::cout << "p1 / 5.0 = " << p1 / 5.0
        << '\n'; // (0.6,0.4)
57
58     // dot / cross via complex
59     std::cout << "dot(p1,p2) = " << dot(p1,
        p2) << '\n';
60     std::cout << "cross(p1,p2) = " << cross(
        p1, p2) << '\n';
61
62     // distances, angle, rotation, polar/
        cartesian
63     std::cout << "squared dist = " << std::
        norm(p1 - p2) << '\n';

```

```

64 std::cout << "euclid dist = " << std::
    abs(p1 - p2) << '\n';
65 std::cout << "angle p1->p2 = " <<
    angle_between(p1, p2) << '\n';
66 std::cout << "angle ABC = " << angle_ABC
    (point(1,0), point(0,0), point(0,1))
    << '\n';

67 // project / reflect / intersect
68 // examples
69 point v(1, 1);
70 point proj = project_onto_vector(p1, v);
71 std::cout << "project p1 onto v = " <<
    proj << '\n';

72 point lineA(0,0), lineB(2,0);
73 std::cout << "project p1 onto line = "
    << project_onto_line(p1, lineA,
74 lineB) << '\n';
75 std::cout << "reflect p1 across line = "
    << reflect_across_line(p1, lineA,
    lineB) << '\n';

76 // intersection example
77 point r1a(0,0), r1b(1,1);
78 point r2a(0,1), r2b(1,0);
79 std::cout << "intersection = " <<
    intersection(r1a, r1b, r2a, r2b) <<
80 '\n';

81 // polar/cartesian and rotation
82 point polar_pt = std::polar(5.0, PI/4);
83 std::cout << "polar(5,PI/4) = " <<
    polar_pt << '\n';
84 point rotated = p1 * std::polar(1.0, PI
    /2); // rotate p1 by 90 degrees
85 // about origin
86 std::cout << "rotate p1 by 90deg = " <<
    rotated << '\n';

87 return 0;
88 }
89 }

```

5.8 point distance from a line

```

1 // calculate area using cross product and
    divide by base length
2 double point_line_distance(const point &p,
    const point &a, const point &b) {
3 // area = 0.5 * |cross(b - a, p - a)|
4 // base = |b - a|
5 // height = 2 * area / base = |cross(b -
    a, p - a)| / |b - a|
6 std::abs(cross(b - a, p - a)) / std::abs
    (b - a);
7 }

```

6 Graph

6.1 Prim

```

1 // Prim's algorithm
2 vector<tuple<int, int, long long>> Prim(
    const vector<vector<pair<int, long long
    >>& adj) {
3 const auto& n = adj.size();
4 vector<tuple<int, int, long long>> mst
    {};

5 vector<bool> found(n, false);
6 using ti = tuple<long long, int, int>;
7 priority_queue<ti, vector<ti>, greater<
    ti>> pq{};
8 found[0] = true;
9 for (auto& [v, w] : adj[0]) pq.emplace(w
    , 0, v);

10 for (int i = 0; i < n - 1; ++i) {
11 int mn, u, v;
12 do {
13 tie(mn, u, v) = pq.top(), pq.pop()
    ();
14 } while (found[v]);
15 found[v] = true, mst.emplace_back(u,
    v, mn);
16 for (auto& [x, w] : adj[v]) pq.
    emplace(w, v, x);
17 }
18 return mst;
19 }
20 }
21 }

```

6.2 Eulerian cycle

```

1 // Eulerian cycle in an undirected graph
2 vector<int> euler_cycle(vector<vector<pair<
    int, int>>& adj, int w = 0) {
3 int n{adj.size()}, m{};
4 for (int v{0}; v < n; ++v) m += adj[v].
    size();
5 m /= 2;

6 vector<int> res{};
7 stack<pair<int, int>> stk{};
8 stk.emplace(w, -1);
9 vector<int> nxt(n);
10 vector<bool> usd(m);
11 while (!stk.empty()) {
12 auto [u, i]{stk.top()};
13 while (nxt[u] < adj[u].size() && usd
    [adj[u][nxt[u].second]] ++nxt[u]
14 );
15 if (nxt[u] < adj[u].size()) {
16 auto [v, j]{adj[u][nxt[u]]};
17 ++nxt[u], usd[j] = true, stk.
    emplace(v, j);
18 } else {
19 }

```

```

20 if (i != -1) res.push_back(i);
21 stk.pop();
22 }
23 return res;
24 }
25 }
26 }
27 int main() {
28 int n = 4; // number of vertices
29 vector<vector<pair<int, int>>> adj(n);
30 // Add edges with edge IDs
31 int eid = 0;
32 auto add_edge = [&](int u, int v) {
33 adj[u].push_back({v, eid});
34 adj[v].push_back({u, eid});
35 ++eid;
36 };
37 add_edge(0, 1);
38 add_edge(1, 2);
39 add_edge(2, 3);
40 add_edge(3, 0);
41 vector<int> cycle = euler_cycle(adj, 0);
42 cout << "Euler cycle (edge IDs in order)
    " << " ";
43 for (int id : cycle) cout << id << " ";
44 cout << "\n";
45 return 0;
46 }
47 }
48 }
49 }
50 }
51 }

```

6.3 maximum weight bipartite matching

```

1 // maximum weight matching in a weighted
    bipartite graph
2 // Hungarian Algorithm - O(V^3)
3 class bipartiteGraph {
4 int n;
5 vector<long long> lx, ly;
6 vector<bool> vx, vy;
7 queue<int> qu{}; // only X's vertices
8 vector<int> py;
9 vector<long long> dy; // smallest (lx[x]
    + ly[y] - w[x][y])
10 vector<int> pdy; // & which x
11 void relax(int x) {
12 for (int y{0}; y < n; ++y)
13 if (lx[x] + ly[y] - w[x][y] < dy
    [y])
14 dy[y] = lx[x] + ly[y] - w[x][y], pdy[y] = x;
15 }
16 void augment(int y) {
17 while (y != -1) {
18 int x{py[y]}, yy{mx[x]};
19 mx[x] = y, my[y] = x;
20 y = yy;
21 }

```

```

22 }
23 bool bfs() {
24 while (!qu.empty()) {
25 int x{qu.front()}; qu.pop();
26 for (int y{0}; y < n; ++y) {
27 if (!vy[y] && lx[x] + ly[y]
    == w[x][y]) {
28 vy[y] = true, py[y] = x;
29 if (my[y] == -1) return
    augment(y), true;
30 int xx{my[y]};
31 qu.push(xx), vx[xx] =
    true, relax(xx);
32 }
33 }
34 return false;
35 }
36 void relabel() {
37 long long d{numeric_limits<long long>
    >::max()};
38 for (int y{0}; y < n; ++y) if (!vy[y]
    ) d = min(d, dy[y]);
39 for (int x{0}; x < n; ++x) if (vx[x]
    ) lx[x] -= d;
40 for (int y{0}; y < n; ++y) if (vy[y]
    ) ly[y] += d;
41 for (int y{0}; y < n; ++y) if (!vy[y]
    ) dy[y] -= d;
42 }
43 bool expand() { // expand one layer with
    new equality edges
44 for (int y{0}; y < n; ++y) {
45 if (!vy[y] && dy[y] == 0) {
46 vy[y] = true, py[y] = pdy[y];
47 if (my[y] == -1) return
    augment(y), true;
48 int xx{my[y]};
49 qu.push(xx), vx[xx] = true,
    relax(xx);
50 }
51 }
52 return false;
53 }
54 public:
55 vector<vector<long long>> w;
56 vector<int> mx, my;
57 bipartiteGraph(int _n) : n{_n}, lx(n),
    ly(n), vx(n), vy(n), py(n), dy(n),
    pdy(n),
58 w(n, vector<long long>(n, 0)), mx(n,
    -1), my(n, -1) {}
59 long long Hungarian() {
60 for (int i{0}; i < n; ++i) {
61 lx[i] = ly[i] = 0;
62 for (int j{0}; j < n; ++j)
63 lx[i] = max(lx[i], w[i][j]);
64 mx[i] = my[i] = -1;
65 }
66 for (int x{0}; x < n; ++x) {
67 fill(vx.begin(), vx.end(), false);
68 fill(vy.begin(), vy.end(), false);
69 }
70 }

```



```

71     fill(dy.begin(), dy.end(),
72           numeric_limits<long long>::
73           max());
74     qu = {}, qu.push(x), vx[x] =
75     true, relax(x);
76     while (!bfs()) {
77         relabel();
78         if (expand()) break;
79     }
80     long long weight{0};
81     for (int x{0}; x < n; ++x) weight +=
82     w[x][mx[x]];
83     return weight;
84 };
85 // usage example:
86 int main() {
87     int n = 4;
88     bipartiteGraph g(n);
89     long long mat[4][4] = {
90         {9, 2, 7, 8},
91         {6, 4, 3, 7},
92         {5, 8, 1, 8},
93         {7, 6, 9, 4}
94     };
95     for (int i = 0; i < n; ++i)
96     for (int j = 0; j < n; ++j)
97         g.w[i][j] = mat[i][j];
98     long long maxWeight = g.Hungarian();
99     std::cout << "Maximum weight: " <<
100     maxWeight << "\n";
101     for (int i = 0; i < n; ++i)
102     std::cout << "X " << i << " matched
103     with Y " << g.mx[i] << "\n";
104 }

```

6.4 maximum flow

```

1 // minimum cut maximum flow
2 // relabel-to-front algorithm
3 // (an implementation of generic push-relabel
4 // algorithm)
5 // handle the capacity, c, of a vertex, v.
6 // Add a new vertex, v', and an edge v-v'
7 // with capacity c.
8 template<typename T> //requires
9 signed_integral<T>
10 void handle_vertex_capacity(int u, T c, int&
11 n, vector<tuple<int, int, T>>& e) {
12     int w{n++};
13     for (auto& [_u, _v, _c] : e) if (_u == u
14         ) _u = w;
15     e.emplace_back(u, w, c);
16 }
17 template<typename T> //requires
18 signed_integral<T>

```

```

15 class flowNetwork {
16     int n, s, t;
17     vector<int> h; // height label
18     vector<T> e; // excess
19     vector<vector<int>>> adj; // adjacent
20     list
21     vector<vector<T>>> c, r;
22     vector<size_t> p; // record the position
23     processed
24     list<int> lst; // topological sort of
25     admissible network
26     void push(int u, int v) {
27         T f{min(e[u], r[u][v])};
28         r[u][v] -= f, r[v][u] += f;
29         e[u] -= f, e[v] += f;
30     }
31     void relabel(int u) {
32         int mn{numeric_limits<int>::max()};
33         for (auto& v : adj[u])
34             if (r[u][v]) mn = min(mn, h[v]);
35         h[u] = mn + 1;
36     }
37     void init() {
38         fill(h.begin(), h.end(), 0);
39         fill(e.begin(), e.end(), 0);
40         lst.clear();
41         for (int u{0}; u < n; ++u) if (u !=
42             s && u != t) lst.push_back(u);
43         fill(p.begin(), p.end(), 0);
44         r = c;
45     }
46     void preflow() {
47         h[s] = n;
48         for (auto& v : adj[s]) {
49             e[v] += r[s][v], e[s] -= r[s][v];
50             r[v][s] += r[s][v], r[s][v] = 0;
51         }
52     }
53     bool discharge(int u) {
54         bool flag{false};
55         while (e[u]) {
56             for (; p[u] < adj[u].size(); ++p
57                 [u]) {
58                 int v{adj[u][p[u]]};
59                 if (r[u][v] && h[u] == h[v]
60                     + 1) push(u, v);
61                 if (!e[u]) break;
62             }
63             if (e[u]) relabel(u), p[u] = 0,
64                 flag = true;
65         }
66         return flag; // return if relabel or
67         not
68     }
69 public:
70     flowNetwork(int _n) : n{_n}, s{0}, t{n -
71         1}, h(n), e(n), adj(n), c(n, vector
72         <T>(n)), p(n) {}
73     void set_st(int _s, int _t) {
74         s = _s, t = _t;
75     }
76     void add_edge(int u, int v, T _c) {
77         if (!c[u][v] && !c[v][u]) adj[u].
78             push_back(v), adj[v].push_back(u)
79             );
80     }

```

```

68     c[u][v] += _c;
69 }
70 T max_flow() {
71     assert(s != t);
72     init();
73     preflow();
74     auto it{lst.begin()};
75     while (it != lst.end()) {
76         if (discharge(*it)) { // relabel
77             -to-front
78             lst.push_front(*it), lst.
79             erase(it);
80             it = lst.begin();
81         }
82         it = next(it);
83     }
84     return e[t];
85 };
86 // Build an example graph with a vertex
87 // capacity on node 2.
88 int n = 4; // nodes: 0 (source), 1, 2, 3
89 (sink)
90 vector<tuple<int, int, T>> edges = {
91     {0, 1, 3},
92     {0, 2, 2},
93     {1, 2, 5},
94     {1, 3, 2},
95     {2, 3, 3}
96 };
97 // Enforce vertex capacity of 2 on node
98 2.
99 handle_vertex_capacity<T>(2, 2, n, edges
100 );
101 // Build the network and compute max
102 flow.
103 flowNetwork<T> net(n);
104 net.set_st(0, 3);
105 for (auto [u, v, cap] : edges) net.
106     add_edge(u, v, cap);
107 cout << "Max flow = " << net.max_flow()
108     << '\n';
109 }

```

6.5 maximum cardinality bipartite matching

```

1 class BipartiteMatching {
2     vector<bool> vis;
3     vector<int> mx{}, my{};

```

```

4     bool dfs(const vector<vector<int>>& adj,
5             int x) {
6         for (int y : adj[x]) {
7             if (vis[y]) continue;
8             vis[y] = true;
9             if (my[y] == -1 || dfs(adj, my[y]
10                 )) {
11                 mx[x] = y, my[y] = x;
12                 return true;
13             }
14         }
15         return false;
16     }
17 public:
18     pair<vector<int>, vector<int>> operator
19     (const vector<vector<int>>& adj,
20     int ny) {
21         vis.assign(ny, false);
22         mx.assign((int)adj.size(), -1);
23         my.assign(ny, -1);
24         for (int x = 0; x < (int)adj.size();
25             ++x) {
26             fill(vis.begin(), vis.end(),
27                 false);
28             dfs(adj, x);
29         }
30         return {mx, my};
31     }
32 // Usage example:
33 int main() {
34     // Left partition has 4 nodes (0..3),
35     // right partition has 4 nodes (0..3).
36     vector<vector<int>> adj(4);
37     adj[0] = {0, 1};
38     adj[1] = {1, 2};
39     adj[2] = {2};
40     adj[3] = {2, 3};
41     BipartiteMatching bm;
42     auto res = bm(adj, 4);
43     const auto& mx = res.first;
44     cout << "Matched pairs (Left -> right):\n";
45     for (int i = 0; i < (int)mx.size(); ++i)
46     if (mx[i] != -1)
47         cout << i << " -> " << mx[i] <<
48         "\n";
49 }

```

6.6 Floyd-Warshall

```

1 // Floyd-Warshall algorithm
2 template<typename T>
3 vector<vector<optional<T>>> Floyd_Warshall(
4     const vector<vector<optional<T>>>& adj)
5 {
6     const auto& n{adj.size()};
7     auto d{adj};
8     for (int i{0}; i < n; ++i) d[i][i] = 0;

```

```

8   for (int k{0}; k < n; ++k)
9       for (int i{0}; i < n; ++i)
10          for (int j{0}; j < n; ++j) {
11              if (!d[i][k] || !d[k][j])
12                  continue; // no value
13              if (!d[i][j] || d[i][j] > d[
14                  i][k].value() + d[k][j].
15                  value())
16                  d[i][j] = d[i][k].value
17                      () + d[k][j].value()
18                      ;
19          }
20      }
21      return d;
22  }

```

6.7 MST

```

1  // Kruskal' s algorithm
2  vector<tuple<int, int, long long>> Kruskal(
3      int n, vector<tuple<long long, int, int
4      >>& edges) {
5      vector<tuple<int, int, long long>> mst
6      {};
7      DSU dsu{n};
8      sort(edges.begin(), edges.end());
9      for (auto& [w, u, v] : edges)
10         if (dsu.find(u) != dsu.find(v))
11             dsu.unionn(u, v, mst.
12                 emplace_back(u, v, w));
13     return mst;
14 }

```

6.8 Hopcroft–Karp

```

1  // maximum cardinality bipartite matching in
2  // unweighted bipartite graph
3  // Hopcroft–Karp algorithm - O(EV)
4  class bipartiteGraph {
5      int nx, ny;
6      vector<int> dx, dy;
7      bool dfs(int y) {
8          for (auto& x : adj[y]) {
9              if (dx[x] + 1 != dy[y]) continue
10             ;
11             if (mx[x] == -1 || dfs(mx[x])) {
12                 my[y] = x, mx[x] = y;
13                 dy[y] = -1; // augmented -
14                 // remove from BFS forest
15                 return true;
16             }
17         }
18         dy[y] = -1; // no augmenting path -
19         // remove from BFS forest
20         return false;
21     }
22     bool bfs() {
23         fill(dx.begin(), dx.end(), -1);
24         fill(dy.begin(), dy.end(), -1);
25     }

```

```

26     queue<int> qu{}, qu2{};
27     for (int x{0}; x < nx; ++x) // use
28         all unmatched x's as roots
29         if (mx[x] == -1) qu.push(x), dx[
30             x] = 0;
31     bool found{false};
32     while (!qu.empty() && !found) { //
33         stop at the level found
34         while (!qu.empty()) {
35             auto x{qu.front()}; qu.pop()
36             ;
37             for (auto& y : adj[x])
38                 if (dy[y] == -1 && my[y]
39                     != x) {
40                     dy[y] = dx[x] + 1;
41                     if (my[y] == -1)
42                         found = true;
43                     else
44                         qu2.push(my[y]);
45                     dx[my[y]] = dy
46                     [y] + 1;
47                 }
48             qu.swap(qu2);
49         }
50         return found;
51     }
52     vector<vector<int>> adjx, adjy;
53     public:
54     vector<int> mx, my;
55     bipartiteGraph(int_nx, int_ny) : nx{
56         _nx}, ny{_ny}, dx(nx), dy(ny)
57         , adjx(nx), adjy(ny), mx(nx, -1), my
58         (ny, -1) {}
59     void addEdge(int x, int y) {
60         adjx[x].push_back(y), adjy[y].
61         push_back(x);
62     }
63     int Hopcroft_Karp() {
64         int c{0};
65         while (bfs()) {
66             for (int y{0}; y < ny; ++y)
67                 if (my[y] == -1 && dy[y] !=
68                     -1) c += dfs(y);
69         }
70         return c;
71     }
72 }

```

// Usage example:

```

1  int main() {
2      // Left partition has 4 nodes (0..3),
3      // right partition has 4 nodes (0..3).
4      bipartiteGraph bg(4, 4);
5      bg.addEdge(0, 0);
6      bg.addEdge(0, 1);
7      bg.addEdge(1, 1);
8      bg.addEdge(1, 2);
9      bg.addEdge(2, 2);
10     bg.addEdge(3, 2);
11     bg.addEdge(3, 3);
12     int max_matching = bg.Hopcroft_Karp();
13     cout << "Maximum matching size: " <<
14         max_matching << "\n";
15 }

```

```

72     cout << "Matched pairs (left -> right):\n";
73     for (int i = 0; i < (int)bg.mx.size();
74         ++i)
75         if (bg.mx[i] != -1)
76             cout << i << " -> " << bg.mx[i]
77             << "\n";
78 }

```

6.9 Dijkstra

```

1  // Dijkstra algorithm
2  template<typename T>
3  vector<optional<T>> Dijkstra(const vector<
4      vector<pair<int, T>>& adj, int s) {
5      const auto& n{adj.size()};
6      vector<optional<T>> d(n, nullopt);
7      d[s] = 0;
8      vector<bool> found(n, false);
9      using pq_t = pair<T, int>;
10     priority_queue<pq_t, vector<pq_t>,
11         greater<pq_t>> pq{};
12     pq.emplace(0, s);
13     while (!pq.empty()) {
14         auto [_, u]{pq.top()}; pq.pop();
15         if (found[u]) continue;
16         found[u] = true;
17         for (auto& [v, w] : adj[u])
18             if (!d[v] || d[v] > d[u].value()
19                 + w) {
20                 d[v] = d[u].value() + w;
21                 pq.emplace(d[v].value(), v);
22             }
23     }
24     return d;
25 }

```

6.10 Dinic

```

1  int n, s, t, cnt, inf = 1e17;
2  vector<vector<int>> adj;
3  int level[100000], ptr[100000], rev[100000],
4  to[100000], cap[100000];
5  void add(int a, int b, int c) {
6      int id = cnt++, rid = cnt++;
7      to[id] = b;
8      to[rid] = a;
9      cap[id] = c;
10     cap[rid] = 0;
11     rev[id] = rid;
12     rev[rid] = id;
13     adj[a].push_back(id);
14     adj[b].push_back(rid);
15 }
16 bool bfs() {
17     memset(level, -1, sizeof(level));
18     queue<int> q;

```

```

19     q.push(s);
20     level[s] = 0;
21     while(!q.empty()) {
22         int curr = q.front(); q.pop();
23         for(int id: adj[curr]) {
24             int next = to[id];
25             if(cap[id] > 0 && level[next] ==
26                 -1) {
27                 level[next] = level[curr] +
28                 1;
29                 q.push(next);
30             }
31         }
32     }
33     return level[t] != -1;
34 }
35 int dfs(int curr, int flow) {
36     if(curr == t || flow == 0) return flow;
37     for(ptr[curr] < adj[curr].size(); ptr[
38         curr]++) {
39         int id = adj[curr][ptr[curr]];
40         int next = to[id];
41         if(cap[id] > 0 && level[next] ==
42             level[curr] + 1) {
43             int temp = dfs(next, min(flow,
44                 cap[id]));
45             if(temp == 0) continue;
46             int rid = rev[id];
47             cap[id] -= temp;
48             cap[rid] += temp;
49             return temp;
50         }
51     }
52     return 0;
53 }
54 int maxflow() {
55     int flow = 0;
56     while(bfs()) {
57         memset(ptr, 0, sizeof(ptr));
58         while(true) {
59             int temp = dfs(s, inf);
60             if(temp == 0) break;
61             flow += temp;
62         }
63     }
64     return flow;
65 }

```

6.11 maximum cardinality matching

```

1  // maximum matching
2  // Edmonds' Blossom algorithm
3  // O(VEa(E,V))
4  class graph {
5      int n;
6      vector<int> p, d, a, c1, c2;
7      // (alternating tree), (unvisited: -1,
8      // even: 0, odd: 1), (auxiliary for lca
9      //), (cross edge)

```

```

8  int label{0};
9  /* DSU */
10 vector<int> dsu_p, dsu_sz, dsu_b;
11 void dsu_reset() {
12     fill(dsu_p.begin(), dsu_p.end(), -1)
13     ;
14     fill(dsu_sz.begin(), dsu_sz.end(), 1);
15     iota(dsu_b.begin(), dsu_b.end(), 0);
16 }
17 int find(int x) { // collapsing find
18     return dsu_p[x] == -1 ? x : dsu_p[x]
19         = find(dsu_p[x]);
20 }
21 int base(int x) { return dsu_b[find(x)];
22 }
23 void contract(int x, int y) { //
24     weighted union
25     auto rx{find(x)}, ry{find(y)};
26     if (rx == ry) return ;
27     auto b{dsu_b[rx]}; // treat x's base
28     as new base
29     if (dsu_sz[rx] < dsu_sz[ry]) swap(rx
30     , ry);
31     dsu_p[ry] = rx, dsu_sz[rx] += dsu_sz
32     [ry], dsu_b[rx] = b;
33 }
34 /*******/
35 queue<int> qu{}; // only even vertices
36 void handle_one_side(int x, int y, int b
37 ) {
38     for (int v{base(x)}; v != b; v =
39         base(p[v])) {
40         c1[v] = x, c2[v] = y, contract(b
41         , v);
42         if (d[v] == 1) qu.push(v); //
43         odd vertex -> even vertex
44     }
45 }
46 int lca(int u, int v) {
47     ++label; // use next number in order
48     not to clear a
49     while (true) {
50         if (a[u] == label) return u;
51         a[u] = label;
52         if (p[u] != -1) u = base(p[u]);
53         swap(u, v);
54     }
55 }
56 void augment(int r, int y) {
57     if (r == y) return ;
58     if (d[y] == 0) { // even vertex ->
59         odd vertex
60         // check d[y] == 0 instead of d[
61         p[y]] == 1, so m[y], y is
62         in the blossom
63         augment(m[y], m[c1[y]]);
64         augment(r, m[c2[y]]);
65         m[c1[y]] = c2[y], m[c2[y]] = c1[
66         y];
67     } else {
68         int x{p[y]};
69         augment(r, m[x]);
70         m[x] = y, m[y] = x;
71     }
72 }

```

```

57 bool bfs(int r) {
58     dsu_reset();
59     fill(d.begin(), d.end(), -1);
60
61     qu = {}, qu.push(r), d[r] = 0;
62     while (!qu.empty()) {
63         int x{qu.front()}; qu.pop();
64         for (auto& y : adj[x]) {
65             if (base(x) == base(y))
66                 continue;
67             if (d[y] == -1) {
68                 p[y] = x, d[y] = 1;
69                 if (m[y] == -1) { //
70                     augmenting path
71                     augment(-1, y);
72                     return true;
73                 } else {
74                     p[m[y]] = y, d[m[y]]
75                     = 0;
76                     qu.push(m[y]);
77                 }
78             } else if (d[base(y)] == 0)
79                 { // blossom
80                     int b{lca(base(x), base(
81                     y))};
82                     handle_one_side(x, y, b)
83                     ;
84                     handle_one_side(y, x, b)
85                     ;
86                 }
87             return false;
88         }
89     }
90     public:
91     vector<vector<int>> adj;
92     vector<int> m;
93     explicit graph(int _n) : n{_n}, p(n, -1)
94     , d(n), a(n), c1(n), c2(n), dsu_p(n)
95     , dsu_sz(n), dsu_b(n), adj(n), m(n, -1) {}
96     int Edmonds() {
97         int c{0};
98         for (int v{0}; v < n; ++v)
99             if (m[v] == -1 && bfs(v)) ++c;
100         return c;
101     }
102 }
103 // usage example:
104 int main() {
105     int n = 5;
106     graph g(n);
107     auto add = [&](int u, int v) {
108         g.adj[u].push_back(v);
109         g.adj[v].push_back(u);
110     };
111     // sample graph
112     add(0, 1);
113     add(0, 2);
114     add(1, 2); // triangle (0,1,2)
115     add(2, 3);
116     add(3, 4);
117     int match_size = g.Edmonds();

```

```

118 cout << "Maximum matching size: " <<
119     match_size << "\n";
120 for (int i = 0; i < n; ++i)
121     if (g.m[i] != -1 && i < g.m[i])
122         cout << i << " - " << g.m[i] <<
123         "\n";
124 }

```

6.12 topological sort

```

1 // topological sort 1
2 optional<vector<int>> top_sort(vector<vector
3 <int>>& adj) {
4     vector<int> res{};
5     int n{static_cast<int>(adj.size())};
6     vector<int> cnt(n, 0); // predecessor
7     count
8     for (int u = 0; u < n; ++u)
9         for (auto& v : adj[u]) ++cnt[v];
10
11     queue<int> qu{};
12     for (int u = 0; u < n; ++u) if (!cnt[u])
13         qu.push(u);
14     while (!qu.empty()) {
15         auto u = qu.front();
16         qu.pop();
17         res.push_back(u);
18
19         for (auto& v : adj[u])
20             if (--cnt[v] == 0) qu.push(v);
21     }
22     if (res.size() != adj.size()) return
23     nullptr;
24     return res;

```

6.13 Floyd's algorithm

```

1 // Floyd's Cycle-Finding Algorithm
2 // Given a function succ(x) that returns the
3 // successor of x,
4 a = succ(x);
5 b = succ(succ(x));
6 while (a != b) {
7     a = succ(a);
8     b = succ(succ(b));
9 }
10 // Find the first element in the cycle
11 a = x;
12 while (a != b) {
13     a = succ(a);
14     b = succ(b);
15 }
16 first = a;
17 // Find the length of the cycle
18 length = 1;
19 b = succ(a);
20 while (b != first) {
21     ++length;
22     b = succ(b);
23 }

```

```

21 while (a != b) {
22     b = succ(b);
23     length = length + 1;
24 }
25 // Finding the starting point of the cycle
26 from any node
27 int n;
28 vector<int> adj, res;
29 vector<vector<int>> radj;
30
31 void fill_radj(int u) {
32     for (int v : radj[u]) {
33         if (res[v] == -1) {
34             res[v] = res[u];
35             fill_radj(v);
36         }
37     }
38 }
39
40 void floyd(int x) {
41     int y = x;
42     do { // find a cycle using Floyd's
43         algorithm
44         x = adj[x];
45         y = adj[adj[y]];
46     } while (y != x);
47     do { // set res[x] = x for all x along
48         cycle
49         res[x] = x;
50         x = adj[x];
51     } while (y != x);
52     do { // set res'es for all x not along
53         cycle
54         fill_radj(x);
55         x = adj[x];
56     } while (y != x);
57 }
58
59 void solve() {
60     cin >> n;
61     adj.resize(n);
62     radj.assign(n, {});
63     res.assign(n, -1);
64
65     for (auto& e : adj) {
66         cin >> e;
67         e--;
68     }
69
70     for (int i = 0; i < n; ++i) {
71         radj[adj[i]].push_back(i);
72     }
73
74     for (int i = 0; i < n; ++i) {
75         if (res[i] == -1) {
76             floyd(i);
77         }
78     }
79
80     for (int i = 0; i < n; ++i) {
81         cout << res[i] + 1 << " ";
82     }
83     cout << "\n";
84 }

```

6.14 stable matching

```

1 // Stable Marriage Problem (stable matching)
2 // Gale-Shapley algorithm
3 class bipartiteGraph {
4 public:
5     int n;
6     vector<vector<int>> px, py;
7     vector<int> mx, my;
8
9     bipartiteGraph(int n) : n(n), px(n,
10        vector<int>(n)), py(n, vector<int>(n
11        )), mx(n, -1), my(n, -1) {}
12 void Gale_Shapley() {
13     queue<int> qu{};
14     vector<priority_queue<pair<int, int>
15        >>> pq(n);
16     for (int x{0}; x < n; ++x) {
17         qu.push(x);
18         for (int y{0}; y < n; ++y) pq[x
19            ].emplace(px[x][y], y);
20     }
21     while (!qu.empty()) {
22         auto x{qu.front()}; qu.pop();
23
24         int y;
25         do {
26             y = pq[x].top().second; pq[x]
27                 .pop();
28             } while (my[y] != -1 && py[y][x]
29                 <= py[y][my[y]]);
30         mx[x] = y, my[y] = x;
31     }
32 }
33 int main() {
34     int n = 3;
35     bipartiteGraph g(n);
36
37     // Men preferences (from most to Least
38     // preferred)
39     vector<vector<int>> men = {
40         {0, 1, 2},
41         {1, 2, 0},
42         {1, 0, 2}
43     };
44
45     // Women preferences (from most to Least
46     // preferred)
47     vector<vector<int>> women = {
48         {1, 0, 2},
49         {2, 1, 0},
50         {0, 1, 2}
51     };
52
53     // Convert orders to scores (higher is
54     // better)
55     for (int x = 0; x < n; ++x)
56         for (int pos = 0; pos < n; ++pos)

```

```

53         g.px[x][men[x][pos]] = n - pos;
54
55     for (int y = 0; y < n; ++y)
56         for (int pos = 0; pos < n; ++pos)
57             g.py[y][women[y][pos]] = n - pos
58                 ;
59     g.Gale_Shapley();
60
61     // Output matches: man x is matched to
62     // woman g.mx[x]
63     for (int x = 0; x < n; ++x)
64         cout << x << " " << g.mx[x] << '\n';
65
66     return 0;
67 }

```

6.15 Bellman-Ford

```

1 // Bellman-Ford algorithm
2 template<typename T>
3 optional<vector<optional<T>>> Bellman_Ford(
4     const vector<vector<pair<int, T>>>& adj,
5     int s) {
6     const auto& n{adj.size()};
7     vector<optional<T>> d(n, nullopt);
8     d[s] = 0;
9
10    queue<int> qu{};
11    vector<bool> in(n, false);
12
13    qu.push(s); in[s] = true;
14    for (int i{0}; i < n; ++i) { // at most
15        // n-1 edges
16        while (!qu.empty()) {
17            int u{qu.front()};
18            qu.pop(); in[u] = false;
19            for (auto& [v, w] : adj[u])
20                if (!d[v] || d[v] > d[u].
21                    value() + w) { // relax
22                    d[v] = d[u].value() + w;
23                    if (!in2[v]) qu2.push(v);
24                    in2[v] = true;
25                }
26            qu.swap(qu2); in.swap(in2);
27        }
28    }
29
30    if (qu.empty()) return d;
31    return nullopt; // if negative cycle
32 }

```

7 Number Theory

7.1 Linear Sieve

```

1 // Calculate the smallest divisor of
2 // integers in [2, maxn] in O(maxn)

```

```

1 vector<int> min_div[] {
2     constexpr int maxn = 400000 + 10;
3
4     vector<int> v(maxn), p;
5
6     for (int i = 2; i < maxn; ++i) {
7         if (!v[i]) {
8             v[i] = i;
9             p.push_back(i);
10        }
11        for (int j = 0; p[j] * i < maxn; ++j)
12            {
13                v[p[j] * i] = p[j];
14                if (p[j] == v[i]) break;
15            }
16        }
17    }
18    return v;
19 }();

```

7.2 C(n,m)

```

1 ll Cnm(ll n, ll m) {
2     if (m > n / 2) m = n - m;
3     ll r{1};
4     for (ll i{1}, j{n}; i <= m; ++i, --j) r
5         *= j, r /= i;
6     return r;
7 }

```

7.3 Euler Totient 1 to n

```

1 void phi_1_to_n(int n) {
2     vector<int> phi(n + 1);
3     for (int i = 0; i <= n; ++i)
4         phi[i] = i;
5
6     for (int i = 2; i <= n; ++i) {
7         if (phi[i] == i) {
8             for (int j = i; j <= n; j += i)
9                 phi[j] -= phi[j] / i;
10        }
11    }
12 }

```

7.4 derangement (Principle of Inclusion-Exclusion)

```

1 // 1. Principle of Inclusion-Exclusion
2 // n! = n! * Σ (from k=0 to n) [((-1)^k) / (
3     // k!)]
4
5 mint c = 1;
6 for (int i = 1; i <= n; ++i) {
7     c = (c * i) + (i % 2 == 1 ? -1 : 1);
8     cout << c.val() << ' ';
9 }

```

7.5 matrix template (with fast power)

```

1 template<class T> struct Matrix {
2     T **mat; int a, b;
3
4     Matrix() : a(0), b(0) {}
5     Matrix(int a_, int b_) : a(a_), b(b_) {
6         int i, j;
7         mat = new T*[a];
8         for (i = 0; i < a; ++i) {
9             mat[i] = new T[b];
10            for (j = 0; j < b; ++j) {
11                mat[i][j] = 0;
12            }
13        }
14    }
15    Matrix(const vector<vector<T>>& vt) {
16        int i, j;
17
18        *this = Matrix((int)vt.size(), (int)
19            vt[0].size());
20        for (i = 0; i < a; ++i) {
21            for (j = 0; j < b; ++j) {
22                mat[i][j] = vt[i][j];
23            }
24        }
25    }
26    Matrix operator*(const Matrix& m) {
27        int i, j, k;
28
29        assert(b == m.a);
30        Matrix r(a, m.b);
31        for (i = 0; i < a; ++i) {
32            for (j = 0; j < m.b; ++j) {
33                for (k = 0; k < b; ++k) {
34                    r.mat[i][j] += mul(mat[i]
35                        [k], m.mat[k][j]);
36                    r.mat[i][j] %= MOD;
37                }
38            }
39        }
40        return r;
41    }
42    Matrix& operator*=(const Matrix& m) {
43        return *this = (*this) * m;
44    }
45    friend Matrix power(Matrix m, long long
46        p) {
47        int i;
48
49        assert(m.a == m.b);
50        Matrix r(m.a, m.b);
51        for (i = 0; i < m.a; ++i) {
52            r.mat[i][i] = 1;
53        }
54        for (; p > 0; p >= 1, m *= m) {
55            if (p & 1) {
56                r *= m;
57            }
58        }
59    }
60 }

```

```

59 int main() {
60     Matrix<int> mat(adj);
61     mat = power(mat, k); // mat^k
62     cout << mat.mat[i][j];
63 }

```

7.6 Sieve of Eratosthenes (with big num)

```

1 const int MX = 100000;
2 bool np[MX + 1];
3 vector<int> prime_numbers;
4
5 int init = []() {
6     np[0] = np[1] = true;
7     for (int i = 2; i <= MX; i++) {
8         if (!np[i]) {
9             prime_numbers.push_back(i);
10            for (int j = i; j <= MX / i; j
11                ++){ // 避免溢出的写法
12                np[i * j] = true;
13            }
14        }
15    }
16    return 0;
17 }();
18
19 bool is_prime(long long n) {
20     if (n <= MX) {
21         return !np[n];
22     }
23     for (long long p : prime_numbers) {
24         if (p * p > n) {
25             break;
26         }
27         if (n % p == 0) {
28             return false;
29         }
30     }
31     return true;

```

7.7 mod inv

```

1 // Modular inverse when mod is prime
2 long long mod_inverse(long long a, long long
3     mod) {
4     return power_mod(a, mod - 2, mod); //
5     Fermat's Little Theorem

```

7.8 derangement (DP)

```

1 // 2. DP
2 // !n = (n - 1) * (! (n - 1) + ! (n - 2)),
3 // with !0 = 1, !1 = 0
4 mint a = 1, b = 0;
5 cout << 0 << ' ';
6
7 for (int i = 2; i <= n; i++) {
8     mint c = (i - 1) * (a + b);
9     cout << c.val() << ' ';
10    a = b;
11    b = c;
12 }

```

7.9 Euler Totient

```

1 int phi(int n) {
2     int result = n;
3     for (int i = 2; i * i <= n; i++) {
4         if (n % i == 0) {
5             while (n % i == 0)
6                 n /= i;
7             result -= result / i;
8         }
9     }
10    if (n > 1)
11        result -= result / n;
12    return result;
13 }

```

7.10 fast power

```

1 /* Iterative Function to calculate (a^b) %
2    mod in O(log b) */
3 long long power_mod(long long a, long long b
4     , long long mod) {
5     long long res{1};
6     while (b > 0) {
7         if (b & 1) res = res * a % mod;
8         b >>= 1;
9         a = (a * a) % mod;
10    }
11    return res;

```

7.11 first and second mex

```

1 // Calculate first and second MEX
2 pair<int, int> calculate_mexes(vector<int>&
3     nums) {
4     int n = nums.size();
5     vector<bool> seen(n + 2, false);
6
7     for (int num : nums) {
8         if (num >= 0 && num < seen.size()) {
9             seen[num] = true;

```

```

9     }
10 }
11
12 int first_mex = -1;
13 int second_mex = -1;
14
15 for (int i = 0; i < seen.size(); ++i) {
16     if (!seen[i]) {
17         if (first_mex == -1) {
18             first_mex = i;
19         } else {
20             second_mex = i;
21             break;
22         }
23     }
24 }
25
26 return {first_mex, second_mex};
27 }

```

7.12 Catalan Number

```

1 // Catalan Number
2 // C(n) = 1/(n+1) * (2n choose n) = (2n)! /
3 // ((n+1)! * n!)
4 ll catalan(int n) {
5     return fact[2 * n] * invf[n + 1] % mod *
6     invf[n] % mod;

```

7.13 Chinese Remainder Theorem

```

1 // coprime (p^k)
2 pair<ll, ll> CRT(const vector<pair<ll, ll>>&
3     congruences) {
4     ll M = 1, sol = 0;
5     for (auto& [m, a] : congruences) M *= m;
6     for (auto& [m, a] : congruences) {
7         ll x = M / m, y = MI(x, m);
8         sol = MA(sol, MM(MM(a, x, M), y, M),
9             M);
10    }
11    return {M, sol};
12
13 // example usage
14 int main() {
15     // x ≡ 2 (mod 3)
16     // x ≡ 3 (mod 5)
17     // x ≡ 2 (mod 7)
18     vector<pair<ll, ll>> congruences = {{3,
19         2}, {5, 3}, {7, 2}};
20     auto [M, sol] = CRT(congruences);
21     cout << "M: " << M << ", x: " << sol <<
22     endl; // M: 105, x: 23
23     return 0;

```

7.14 mod inv (not prime)

```

1 /* exists when a and mod are coprime */
2 /* but mod is not prime */
3 long long MI(long long a, long long mod) {
4     return power_mod(a, euler_phi(mod) - 1,
5         mod);

```

7.15 mod inv (not coprime)

```

1 /* a and mod are not coprime */
2 long long MI(long long a, long long mod) {
3     long long d, x, y;
4     extEcu(a, mod, d, x, y);
5     return d == 1 ? (x + mod) % mod : -1;
6 }

```

7.16 inclusion-exclusion principle

```

1 /*
2 inclusion-exclusion principle:  $|U \setminus A| = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots$ 
3
4 if  $f(n) = \sum g(d)$  for all  $d|n$ 
5 then  $g(n) = \sum \mu(d) * f(n/d)$  for all  $d|n$ 
6
7 where  $\mu(d)$  is the mobius function defined as :
8
9  $|U \setminus A| = \sum \mu(d) * |A_d|$ 
10 where  $A_d = \{x \in U : d|x\}$ 
11  $\mu(d) = 1$  if  $d$  is a product of even number of
12 distinct primes
13  $\mu(d) = -1$  if  $d$  is a product of odd number of
14 distinct primes
15  $\mu(d) = 0$  if  $d$  has a squared prime factor
16
17 /*
18 problem: In number theory we call an integer
19 square-free if it is not divisible by a
20 perfect square, except 1. You have to
21 count them!
22 square-free number: An integer  $n$  is square-
23 free if no  $p^2$  divides  $n$ .
24
25 */
26 const int VALMAX = 1e7;
27
28 int mobius[VALMAX];
29
30 int main() {
31     int test_num;
32     cin >> test_num;
33
34     mobius[1] = -1;
35     for (int i = 1; i < VALMAX; i++) {
36         if (mobius[i]) {

```



```

31 mobius[i] = -mobius[i];
32 for (int j = 2 * i; j < VALMAX; j += i) { mobius[j] += mobius[i]; }
33 }
34 }
35
36 for (int t = 0; t < test_num; t++) {
37     long long n;
38     long long ans = 0;
39     cin >> n;
40     for (int i = 1; 1LL * i * i <= n; i++) {
41         ans += mobius[i] * n / ((long long)i * i);
42     }
43     cout << ans << '\n';
44 }
45 }
46
47 /*
48 problem: Given N cows (2 <= N <= 50,000),
49         each listing five favorite ice cream
50         flavors,
51         where two cows are compatible if they share
52         at least one flavor,
53         determine the number of cow pairs that are
54         not compatible.
55
56 ans = n * (n - 1) / 2 - Σ |A_i| + Σ |A_i n
57         A_j| - Σ |A_i n A_j n A_k| + ...
58 where A_i is the set of cows that like
59         flavor i
60 */
61
62 void solve() {
63     int n;
64     cin >> n;
65     vector<array<int, 5>> vc(n);
66
67     map<vector<int>, int> subsets;
68     for (int i = 0; i < n; ++i) {
69         for (int j = 0; j < 5; ++j) {
70             cin >> vc[i][j];
71         }
72
73         sort(vc[i].begin(), vc[i].end());
74         for (int mask = 1; mask < (1<<5); ++mask) {
75             vector<int> subset;
76             for (int b = 0; b < 5; ++b) {
77                 if (mask & (1<<b)) {
78                     subset.push_back(vc[i][b]);
79                 }
80             }
81             subsets[subset]++;
82         }
83     }
84
85     int ans = n * (n - 1) / 2;
86     for (const auto& [subset, freq] : subsets) {
87         ans -= (subset.size() % 2 == 1 ? 1 : -1) * freq * (freq - 1) / 2;
88     }
89     cout << ans << "\n";
90 }

```

7.17 C(n,k) mod inverse

```

1 fac[0] = 1;
2 for (int i = 1; i <= n; ++i) {
3     fac[i] = fac[i - 1] * i % MOD;
4 }
5
6 inv_fac[n] = power_mod(fac[n], MOD - 2, MOD);
7
8 for (int i = n - 1; i >= 0; --i) {
9     inv_fac[i] = inv_fac[i + 1] * (i + 1) % MOD;
10 }
11
12 // C(n, k) = fac[n] * inv_fac[k] * inv_fac[n - k];

```

7.18 Linear Diophantine 丢番圖

```

1 // ax + by = c
2 // x0 = x * (c / gcd(a, b))
3 // y0 = y * (c / gcd(a, b))
4 // x = x0 + k * (b / gcd(a, b))
5 // y = y0 - k * (a / gcd(a, b))
6 // k is any integer
7
8 t11 extendedEuclid(ll a, ll b) {
9     if (b == 0) return {a, 1, 0};
10    ll d, x1, y1;
11    tie(d, x1, y1) = extendedEuclid(b, a % b);
12    return {d, y1, x1 - (a / b) * y1};
13 }
14
15 p11 linearDiophantine(ll a, ll b, ll c) {
16    ll d, x, y;
17    tie(d, x, y) = extendedEuclid(a, b);
18    if (c % d != 0) return {-1, -1}; // no solution
19    x *= c / d;
20    y *= c / d;
21    return {x, y};
22 }
23
24 // Example usage
25 int main() {
26    ll a = 15, b = 10, c = 35;
27    p11 res = linearDiophantine(a, b, c);
28    if (res.first == -1 && res.second == -1) {
29        cout << "No solution exists" << endl;
30    } else {
31        cout << "One solution is x = " << res.first << ", y = " << res.second << endl;
32    }
33    return 0;
34 }
35
36 ax + by = c

```

7.19 擴展歐基里德

```

1 /* solve x, y for ax + by = gcd(a, b) = g */
2 template<typename T>
3 void extEcu(T a, T b, T &g, T &x, T &y) {
4     if (b) extEcu(b, a % b, g, y, x), y -= x * (a / b);
5     else g = a, x = 1, y = 0;
6 }

```

7.20 Sieve of Eratosthenes

```

1 void sieve(vector<int>& primes) {
2     vector<int> is_prime(INF + 1, 0);
3     is_prime[0] = 1;
4     is_prime[1] = 1;
5
6     int sq = sqrt(INF);
7
8     for (int i = 2; i <= sq; ++i) {
9         if (!is_prime[i]) {
10            primes.push_back(i);
11            for (int j = i * i; j <= INF; j += i) {
12                is_prime[j] = 1;
13            }
14        }
15    }
16 }

```

7.21 builtin

```

1 // __builtin_ctz
2 // 這個函數作用是返回輸入數二進制表示從最低
3 // 位開始 (右起) 的連續的0的個數; 如果傳入0
4 // 則行為未定義。
5 int __builtin_ctz (unsigned int x)
6
7 // __builtin_clz
8 // 這個函數作用是返回輸入數二進制表示從最高
9 // 位開始 (左起) 的連續的0的個數; 如果傳入0
10 // 則行為未定義。
11 int __builtin_clz (unsigned int x)
12
13 // __builtin_ffs
14 // 這個函數作用是返回輸入數二進制表示的最低
15 // 非0位的位置, 下標從1開始計數; 如果傳入0
16 // 則返回0
17 int __builtin_ffs (unsigned int x)
18
19 // __builtin_popcount
20 // 這個函數作用是返回輸入數二進制表示中1的個數。
21 int __builtin_popcount (unsigned int x)
22
23 // __builtin_parity

```

```

18 // 這個函數作用是返回輸入數二進制表示中1的個
19 // 數的奇偶性, 1表示奇數個1, 0表示偶數個1。
20 int __builtin_parity (unsigned int x)

```

7.22 C(n,k) DP

```

1 long long binomial(long long n, long long k,
2 long long p) {
3     // dp[i][j] = iCj
4     vector<vector<long long>> dp(n + 1,
5 vector<long long>(k + 1, 0));
6
7     for (int i = 0; i <= n; ++i) {
8         dp[i][0] = 1;
9         if (i <= k) dp[i][i] = 1;
10    }
11
12    for (int i = 0; i <= n; ++i) {
13        for (int j = 1; j <= min(i, k); ++j) {
14            if (i != j) {
15                dp[i][j] = (dp[i - 1][j - 1] + dp[i - 1][j]) % p;
16            }
17        }
18    }
19    return dp[n][k];
20 }

```

8 Range Queries

8.1 subarray maximum sum queries

```

1 // CSES: Subarray Sum Queries
2
3 const int maxn = 2e5;
4 int n = 0;
5 int t[maxn << 1];
6
7 void build() {
8     for (int i = 0; i < n; ++i) {
9         t[i + n] = 0;
10    }
11    for (int i = n - 1; i >= 1; --i) {
12        t[i] = t[i << 1] + t[i << 1 | 1];
13    }
14 }
15
16 void update(int idx, int val) {
17     idx += n;
18     if (t[idx] == val) return;
19     t[idx] = val;
20     for (idx >= 1; idx >= 1; idx >= 1) {
21         t[idx] = t[idx << 1] + t[idx << 1 | 1];
22     }
23 }

```

```

24 int query(int l, int r) {
25     int ans = 0;
26     for (l += n, r += n; l < r; l >= 1, r
27         >= 1) {
28         if (l & 1) ans += t[l++];
29         if (r & 1) ans += t[--r];
30     }
31     return ans;
32 }
33
34 void solve() {
35     int q;
36     cin >> q;
37     n = 0;
38     vector<pii> op(q);
39     map<int, int> mp;
40     vector<int> to_val;
41
42     memset(t, 0, sizeof(t));
43
44     for (int i = 0; i < q; ++i) {
45         char c;
46         cin >> c >> op[i].second;
47         if (c == 'I') {
48             op[i].first = 0;
49             mp[op[i].second] = 1;
50         }
51         else if (c == 'D') {
52             op[i].first = 1;
53             mp[op[i].second] = 1;
54         }
55         else if (c == 'K') {
56             op[i].first = 2;
57         }
58         else {
59             op[i].first = 3;
60         }
61     }
62
63     for (auto& x : mp) {
64         x.second = n;
65         to_val.push_back(x.first);
66         n++;
67     }
68
69     build();
70
71     for (pii o : op) {
72         if (o.first == 0) {
73             int idx = mp[o.second];
74             update(idx, 1);
75         }
76         if (o.first == 1) {
77             int idx = mp[o.second];
78             update(idx, 0);
79         }
80         if (o.first == 2) {
81             int k = o.second;
82             int ans = -1;
83             int l = 0, r = n - 1;
84             while (l <= r) {
85                 int m = l + (r - l) / 2;
86                 int rk = query(0, m + 1);
87                 if (rk >= k) {
88                     ans = m;

```

```

89         r = m - 1;
90     }
91     else {
92         l = m + 1;
93     }
94 }
95 if (ans == -1) cout << "invalid\n";
96 else cout << to_val[ans] << "\n";
97 }
98 if (o.first == 3) {
99     auto it = mp.lower_bound(o.
100         second);
101     if (it == mp.begin() && (it ==
102         mp.end() || o.second <= it->
103         first)) {
104         cout << "0\n";
105     }
106     else if (it == mp.end()) {
107         cout << query(0, n) << "\n";
108     }
109     else {
110         cout << query(0, it->second)
111             << "\n";
112     }
113 }
114 }
115 }

```

8.2 find first equal or greater

```

1 // CSE: Hotel Queries
2
3 int n, m;
4 int sz = 1;
5 const int maxn = 2e6;
6 int t[maxn << 1];
7
8 void build(const vector<int>& a) {
9     for (int i = 0; i < n; ++i) {
10         t[i + sz] = a[i];
11     }
12     for (int i = sz - 1; i >= 1; --i) {
13         t[i] = max(t[i << 1], t[i << 1 | 1]);
14     }
15 }
16
17 void update(int i, int val) {
18     i += sz;
19     t[i] = val;
20     for (i >= 1; i >= 1; i >= 1) {
21         t[i] = max(t[i << 1], t[i << 1 | 1]);
22     }
23 }
24
25 int query(int r) {
26     if (t[1] < r) return 0;
27     int i = 1;

```

```

29 while (i < sz) {
30     if (t[i << 1] >= r) i = i << 1;
31     else i = i << 1 | 1;
32 }
33 return i - sz + 1;
34 }
35
36 void solve() {
37     cin >> n >> m;
38     vector<int> a(n);
39
40     for (int i = 0; i < n; ++i) {
41         cin >> a[i];
42     }
43
44     while (sz < n) sz <= 1;
45     memset(t, 0, sizeof(t));
46     build(a);
47
48     int r;
49     while (m--) {
50         cin >> r;
51         int idx = query(r);
52         cout << idx << " ";
53         if (idx) {
54             a[idx - 1] -= r;
55             update(idx - 1, a[idx - 1]);
56         }
57     }
58     cout << "\n";
59 }

```

8.3 find k-th and count less than

```

1 /*
2 SPOJ: ORDERSET - Order statistic set
3
4 In this problem, you have to maintain a
5 dynamic set of numbers which support the
6 two fundamental operations
7
8 INSERT(S,x): if x is not in S, insert x into
9 S
10 DELETE(S,x): if x is in S, delete x from S
11 and the two type of queries
12
13 K-TH(S) : return the k-th smallest element
14 of S
15 COUNT(S,x): return the number of elements of
16 S smaller than x
17
18 */
19
20 const int maxn = 2e5;
21 int n = 0;
22 int t[maxn << 1];
23
24 void build() {
25     for (int i = 0; i < n; ++i) {
26         t[i + n] = 0;
27     }
28     for (int i = n - 1; i >= 1; --i) {
29         t[i] = t[i << 1] + t[i << 1 | 1];
30     }

```

```

31 }
32
33 void update(int idx, int val) {
34     idx += n;
35     if (t[idx] == val) return;
36     t[idx] = val;
37     for (idx >= 1; idx >= 1; idx >= 1) {
38         t[idx] = t[idx << 1] + t[idx << 1 |
39             1];
40     }
41 }
42
43 int query(int l, int r) {
44     int ans = 0;
45     for (l += n, r += n; l < r; l >= 1, r
46         >= 1) {
47         if (l & 1) ans += t[l++];
48         if (r & 1) ans += t[--r];
49     }
50     return ans;
51 }
52
53 void solve() {
54     int q;
55     cin >> q;
56     n = 0;
57     vector<pii> op(q);
58     map<int, int> mp;
59     vector<int> to_val;
60
61     memset(t, 0, sizeof(t));
62
63     for (int i = 0; i < q; ++i) {
64         char c;
65         cin >> c >> op[i].second;
66         if (c == 'I') {
67             op[i].first = 0;
68             mp[op[i].second] = 1;
69         }
70         else if (c == 'D') {
71             op[i].first = 1;
72             mp[op[i].second] = 1;
73         }
74         else if (c == 'K') {
75             op[i].first = 2;
76         }
77         else {
78             op[i].first = 3;
79         }
80     }
81
82     for (auto& x : mp) {
83         x.second = n;
84         to_val.push_back(x.first);
85         n++;
86     }
87
88     build();
89
90     for (pii o : op) {
91         if (o.first == 0) {
92             int idx = mp[o.second];
93             update(idx, 1);
94         }
95         if (o.first == 1) {
96             int idx = mp[o.second];

```

```

89     update(idx, 0);
90 }
91 if (o.first == 2) {
92     int k = o.second;
93     int ans = -1;
94     int l = 0, r = n - 1;
95     while (l <= r) {
96         int m = l + (r - l) / 2;
97         int rk = query(0, m + 1);
98         //printf("(m, rk): %lld, %
99         //lld\n", m, rk);
100         if (rk >= k) {
101             ans = m;
102             r = m - 1;
103         }
104         else {
105             l = m + 1;
106         }
107     }
108     if (ans == -1) cout << "invalid\
109     n";
110     else cout << to_val[ans] << "\n"
111     ;
112 }
113 if (o.first == 3) {
114     auto it = mp.lower_bound(o.
115     second);
116     if (it == mp.begin() && (it ==
117     mp.end() || o.second <= it->
118     first)) {
119         cout << "0\n";
120     }
121     else if (it == mp.end()) {
122         cout << query(0, n) << "\n";
123     }
124     else {
125         cout << query(0, it->second)
126         << "\n";
127     }
128 }
129 }
130 }

```

9 String

9.1 Z

```

1 // 計算并返回 z 数组 · 其中 z[i] = |LCP(s[i
2 :], s)|
3 vector<int> calc_z(const string& s) {
4     int n = s.size();
5     vector<int> z(n);
6     int box_l = 0, box_r = 0;
7     for (int i = 1; i < n; i++) {
8         if (i <= box_r) {
9             z[i] = min(z[i - box_l], box_r -
10             i + 1);
11         }
12         while (i + z[i] < n && s[z[i]] == s[
13             i + z[i]]) {
14         }
15     }
16 }

```

```

11     box_l = i;
12     box_r = i + z[i];
13     z[i]++;
14 }
15 }
16 z[0] = n;
17 return z;
18 }

```

9.2 manacher

```

1 class Solution {
2 public:
3     int countSubstrings(string s) {
4         int l1 = s.size(), l2 = l1 * 2 + 1;
5         string ch = "#";
6         for(char c: s) {
7             ch = ch + c + "#";
8         }
9         int c = 0, r = 0, cnt = 0;
10        vector<int> p(l2);
11        for(int i = 0; i < l2; i++) {
12            p[i] = (i < r)? min(p[2 * c - i
13            ], r - i): 1;
14            while (i + p[i] < l2 && i - p[i]
15            >= 0 && ch[i + p[i]] == ch[i
16            - p[i]]) p[i]++;
17            if(i + p[i] > r) {
18                r = i + p[i];
19                c = i;
20            }
21            int l = p[i] - 1;
22            if(l % 2 == 0) cnt += 1 / 2;
23            else cnt += 1 / 2 + 1;
24        }
25        return cnt;
26    }
27 };

```

9.3 AC 自動機

```

1 template<char L='a',char R='z'>
2 class ac_automaton{
3     struct joe{
4         int next[R-L+1],fail,efl,ed,cnt_dp,vis;
5         joe():ed(0),cnt_dp(0),vis(0){
6             for(int i=0;i<=R-L;++i)next[i]=0;
7         }
8     };
9     public:
10        std::vector<joe> S;
11        std::vector<int> q;
12        int qs,qe,vt;
13        ac_automaton():S(1),qs(0),qe(0),vt(0){
14            void clear(){
15                q.clear();
16                S.resize(1);
17                for(int i=0;i<=R-L;++i)S[0].next[i]=0;
18                S[0].cnt_dp=S[0].vis=qs=qe=vt=0;
19            }
20        }
21 };

```

```

19 }
20 void insert(const char *s){
21     int o=0;
22     for(int i=0,id;s[i];++i){
23         id=s[i]-L;
24         if(!S[o].next[id]){
25             S.push_back(joe());
26             S[o].next[id]=S.size()-1;
27         }
28         o=S[o].next[id];
29     }
30     ++S[o].ed;
31 }
32 void build_fail(){
33     S[0].fail=S[0].efl=-1;
34     q.clear();
35     q.push_back(0);
36     ++qe;
37     while(qs!=qe){
38         int pa=q[qs++],id,t;
39         for(int i=0;i<=R-L;++i){
40             t=S[pa].next[i];
41             if(!t)continue;
42             id=S[pa].fail;
43             while(~id&&S[id].next[i])id=S[id].
44             fail;
45             S[t].fail=~id?S[id].next[i]:0;
46             S[t].efl=S[S[t].fail].ed?S[t].fail:S
47             [S[t].fail].efl;
48             q.push_back(t);
49             ++qe;
50         }
51     }
52     /*DP出每個前綴在字串s出現的次數並傳回所有
53     字串被s匹配成功的次數O(N*M)*/
54     int match_0(const char *s){
55         int ans=0,id,p=0,i;
56         for(i=0;s[i];++i){
57             id=s[i]-L;
58             while(!S[p].next[id]&&p=S[p].fail;
59             if(!S[p].next[id])continue;
60             p=S[p].next[id];
61             ++S[p].cnt_dp; /*匹配成功則它所有後綴都
62             可以被匹配(DP計算)*/
63         }
64         for(i=qe-1;i>=0;--i){
65             ans+=S[q[i]].cnt_dp*S[q[i]].ed;
66             if(~S[q[i]].fail)S[S[q[i]].fail].
67             cnt_dp+=S[q[i]].cnt_dp;
68         }
69         return ans;
70     }
71     /*多串匹配走efl邊並傳回所有字串被s匹配成功
72     的次數O(N*M^1.5)*/
73     int match_1(const char *s)const{
74         int ans=0,id,p=0,t;
75         for(int i=0;s[i];++i){
76             id=s[i]-L;
77             while(!S[p].next[id]&&p=S[p].fail;
78             if(!S[p].next[id])continue;
79             p=S[p].next[id];
80             if(S[p].ed)ans+=S[p].ed;
81             for(t=S[p].efl;~t;t=S[t].efl){
82                 ans+=S[t].ed; /*因為都走efl邊所以保證
83                 匹配成功*/
84             }
85         }
86         return ans;
87     }
88     /*把AC自動機變成真的自動機*/
89     void evolution(){
90         for(qs=1;qs!=qe;){
91             int p=q[qs++];
92             for(int i=0;i<=R-L;++i)
93                 if(S[p].next[i]==0)S[p].next[i]=S[S[p].
94                 fail].next[i];
95         }
96     }
97 };

```

```

77     ans+=S[t].ed; /*因為都走efl邊所以保證
78     匹配成功*/
79 }
80 }
81 return ans;
82 }
83 /*枚舉(s的子字串nA)的所有相異字串各恰一次
84 並傳回次數O(N*M^(1/3))*/
85 int match_2(const char *s){
86     int ans=0,id,p=0,t;
87     ++vt;
88     /*把載記vt+=1 · 只要vt沒溢位 · 所有S[p].
89     vis=vt就會變成false
90     這種利用vt的方法可以O(1)歸零vis陣列*/
91     for(int i=0;s[i];++i){
92         id=s[i]-L;
93         while(!S[p].next[id]&&p=S[p].fail;
94         if(!S[p].next[id])continue;
95         p=S[p].next[id];
96         if(S[p].ed&&S[p].vis!=vt){
97             S[p].vis=vt;
98             ans+=S[p].ed;
99         }
100         for(t=S[p].efl;~t&&S[t].vis!=vt;t=S[t]
101         .efl){
102             S[t].vis=vt;
103             ans+=S[t].ed; /*因為都走efl邊所以保證
104             匹配成功*/
105         }
106     }
107     return ans;
108 }
109 }
110 }
111 }
112 };

```

9.4 KMP

```

1 // 在文本串 text 中查找模式串 pattern · 返回
2 所有成功匹配的位置 (pattern[0] 在 text
3 中的下标)
4 vector<int> kmp(const string& text, const
5 string& pattern) {
6     int m = pattern.size();
7     vector<int> pi(m);
8     int cnt = 0;
9     for (int i = 1; i < m; i++) {
10         char b = pattern[i];
11         while (cnt && pattern[cnt] != b) {
12             cnt = pi[cnt - 1];
13         }
14         if (pattern[cnt] == b) {
15             cnt++;
16         }
17     }
18 }

```

```

14     pi[i] = cnt;
15 }
16
17 vector<int> pos;
18 cnt = 0;
19 for (int i = 0; i < text.size(); i++) {
20     char b = text[i];
21     while (cnt && pattern[cnt] != b) {
22         cnt = pi[cnt - 1];
23     }
24     if (pattern[cnt] == b) {
25         cnt++;
26     }
27     if (cnt == m) {
28         pos.push_back(i - m + 1);
29         cnt = pi[cnt - 1];
30     }
31 }
32 return pos;
33 }

```

9.5 suffix array lcp

```

1 #define radix_sort(x,y){\
2     for(i=0;i<A;++i)c[i]=0;\
3     for(i=0;i<n;++i)c[x[y[i]]]++;\
4     for(i=1;i<A;++i)c[i]+=c[i-1];\
5     for(i=n-1;~i;--i)sa[--c[x[y[i]]]]=y[i];\
6 }
7 #define AC(r,a,b)\
8     r[a]!=r[b]||a+k>n||r[a+k]!=r[b+k]
9 void suffix_array(const char *s,int n,int *
10     sa,int *rank,int *tmp,int *c){
11     int A='z'+1,i,k,id=0;
12     for(i=0;i<n;++i)rank[tmp[i]]=s[i];
13     radix_sort(rank,tmp);
14     for(k=1;id<n-1;k<=1){
15         for(id=0,i=n-k;i<n;++i)tmp[id++]=i;
16         for(i=0;i<n;++i)
17             if(sa[i]>=k)tmp[id++]=sa[i]-k;
18         radix_sort(rank,tmp);
19         swap(rank,tmp);
20         for(rank[sa[0]]=id=0,i=1;i<n;++i)
21             rank[sa[i]]=id+=AC(tmp,sa[i-1],sa[i]);
22         A=id+1;
23     }
24 }
25 //h:高度數組 sa:後綴數組 rank:排名
26 void suffix_array_lcp(const char *s,int len,
27     int *h,int *sa,int *rank){
28     for(int i=0;i<len;++i)rank[sa[i]]=i;
29     for(int i=0,k=0;i<len;++i){
30         if(rank[i]==0)continue;
31         if(k)--k;
32         while(s[i+k]==s[sa[rank[i]-1]+k])++k;
33         h[rank[i]]=k;
34     }
35     h[0]=0; // h[k]=Lcp(sa[k],sa[k-1]);
36 }

```

9.6 hash

```

1 #define MAXN 1000000
2 #define mod 1073676287
3 /*mod 必須要是質數*/
4 typedef long long T;
5 char s[MAXN+5];
6 T h[MAXN+5]; /*hash陣列*/
7 T h_base[MAXN+5]; /*h_base[n]=(prime^n)%mod*/
8 void hash_init(int len,T prime){
9     h_base[0]=1;
10    for(int i=1;i<=len;++i){
11        h[i]=(h[i-1]*prime+s[i-1])%mod;
12        h_base[i]=(h_base[i-1]*prime)%mod;
13    }
14 }
15 T get_hash(int l,int r){/*閉區間寫法·設編號
16     為0 ~ Len-1*/
17     return (h[r+1]-(h[l]*h_base[r-l+1])%mod+
18         mod)%mod;
19 }

```

9.7 minimal string rotation

```

1 int min_string_rotation(const string &s){
2     int n=s.size(),i=0,j=1,k=0;
3     while(i<n&&j<n&k<n){
4         int t=s[(i+k)%n]-s[(j+k)%n];
5         ++k;
6         if(t){
7             if(t>0)i+=k;
8             else j+=k;
9             if(i==j)++j;
10            k=0;
11        }
12    }
13    return min(i,j); //最小循環表示法起始位置
14 }

```

9.8 reverseBWT

```

1 const int MAXN = 305, MAXC = 'Z';
2 int ranks[MAXN], tots[MAXC], first[MAXC];
3 void rankBWT(const string &bw){
4     memset(ranks,0,sizeof(int)*bw.size());
5     memset(tots,0,sizeof(tots));
6     for(size_t i=0;i<bw.size();++i)
7         ranks[i] = tots[int(bw[i])]+1;
8 }
9 void firstCol(){
10    memset(first,0,sizeof(first));
11    int totc = 0;
12    for(int c='A';c<='Z';++c){
13        if(!tots[c]) continue;
14        first[c] = totc;
15        totc += tots[c];
16    }
17 }

```

```

18 string reverseBwt(string bw,int begin){
19     rankBWT(bw, firstCol());
20     int i = begin; //原字串最後一個元素的位置
21     string res;
22     do{
23         char c = bw[i];
24         res = c + res;
25         i = first[int(c)] + ranks[i];
26     }while (i != begin );
27     return res;
28 }

```

10 Tree

10.1 Euler tour+RMQ

```

1 // Euler Tour Technique
2 class LCA {
3     const vector<vector<int>>& adj;
4     int n;
5     vector<int> d, first, euler{}, log2{};
6     vector<vector<int>> st{};
7     void dfs(int u, int w = -1, int dep = 0) {
8         d[u] = dep;
9         first[u] = euler.size();
10        euler.push_back(u);
11        for (auto& v : adj[u]) {
12            if (v == w) continue;
13            dfs(v, u, dep + 1);
14            euler.push_back(u);
15        }
16    }
17 public:
18     LCA(const vector<vector<int>>& _adj, int
19         root) : adj{_adj}, n{adj.size()}, d
20         (n), first(n) {
21         dfs(root);
22     }
23     int tn{euler.size()};
24     log2.resize(tn + 1);
25     log2[1] = 0;
26     for (int i{2}; i <= tn; ++i) log2[i]
27         = log2[i / 2] + 1;
28
29     st.assign(tn, vector<int>(log2[tn] +
30         1));
31     for (int i{tn - 1}; i >= 0; --i) {
32         st[i][0] = euler[i];
33         for (int j{1}; i + (1 << j) <=
34             tn; ++j) {
35             auto& x{st[i][j - 1]};
36             auto& y{st[i + (1 << (j - 1))][j - 1]};
37             st[i][j] = d[x] <= d[y] ? x
38                 : y;
39         }
40     }
41 }
42 int operator()(int u, int v) {
43     int l{first[u]}, r{first[v]};
44 }

```

```

38     if (l > r) swap(l, r);
39     ++r; // make the interval left
40         closed right open
41
42     int j{log2[r - l]};
43     auto& x{st[l][j]};
44     auto& y{st[r - (1 << j)][j]};
45     return d[x] <= d[y] ? x : y;
46 }
47
48 int main() {
49     int n, q;
50     cin >> n >> q;
51     vector<vector<int>> adj(n);
52
53     for (int i = 1; i < n; ++i) {
54         int u;
55         cin >> u;
56         u--;
57         adj[u].pb(i);
58         adj[i].pb(u);
59     }
60
61     LCA lca(adj, 0);
62
63     while (q--) {
64         int u, v;
65         cin >> u >> v;
66         u--, v--;
67         cout << lca(u, v) + 1 << "\n";
68     }
69     return 0;
70 }

```

10.2 HLD template

```

1 struct HLD {
2     int n, cur = 0;
3     vector<int> sz, top, dep, par, tin, tout
4         , seq;
5     vector<vector<int>> adj;
6     HLD(int n = 0) {
7         init(n);
8     }
9     void init(int n_) {
10        n = n_;
11        sz.assign(n, 1);
12        top.assign(n, -1);
13        dep.assign(n, 0);
14        par.assign(n, -1);
15        tin.assign(n, 0);
16        tout.assign(n, 0);
17        seq.assign(n, 0);
18        adj.resize(n, {});
19    }
20    void add_edge(int u, int v) { adj[u].
21        push_back(v), adj[v].push_back(u); }
22    void build(int root = 0) {
23        top[root] = root, dep[root] = 0, par
24            [root] = -1;
25        dfs1(root), dfs2(root);
26    }
27 }

```

```

24 void dfs1(int u) {
25     if (auto it = find(adj[u].begin(),
26         adj[u].end(), par[u]); it != adj
27         [u].end()) {
28         adj[u].erase(it);
29     }
30     for (auto &v : adj[u]) {
31         par[v] = u;
32         dep[v] = dep[u] + 1;
33         dfs1(v);
34         sz[u] += sz[v];
35         if (sz[v] > sz[adj[u][0]]) {
36             swap(v, adj[u][0]);
37         }
38     }
39 void dfs2(int u) {
40     tin[u] = cur++;
41     seq[tin[u]] = u;
42     for (auto v : adj[u]) {
43         top[v] = v == adj[u][0] ? top[u]
44             : v;
45         dfs2(v);
46     }
47     tout[u] = cur;
48 }
49 int lca(int u, int v) {
50     while (top[u] != top[v]) {
51         if (dep[top[u]] > dep[top[v]]) {
52             u = par[top[u]];
53         } else {
54             v = par[top[v]];
55         }
56     }
57     return dep[u] < dep[v] ? u : v;
58 }
59 int dist(int u, int v) { return dep[u] +
60     dep[v] - 2 * dep[lca(u, v)]; }
61 int jump(int u, int k) {
62     if (dep[u] < k) { return -1; }
63     int d = dep[u] - k;
64     while (dep[top[u]] > d) { u = par[
65         top[u]]; }
66     return seq[tin[u] - dep[u] + d];
67 }
68 // u is v's ancestor
69 bool is_ancestor(int u, int v) { return
70     tin[u] <= tin[v] && tin[v] < tout[
71     u]; }
72 // root's parent is itself
73 int rooted_parent(int r, int u) {
74     if (r == u) { return u; }
75     if (is_ancestor(r, u)) { return par[
76         u]; }
77     auto it = upper_bound(adj[u].begin(),
78         adj[u].end(), r, [&](int x,
79         int y) {
80             return tin[x] < tin[y];
81         }) - 1;
82     return *it;
83 }
84 // rooted at u, v's subtree size
85 int rooted_size(int r, int u) {
86     if (r == u) { return n; }
87     if (is_ancestor(u, r)) { return sz[u]
88         ; }
89     return n - sz[rooted_parent(r, u)];
90 }

```

```

78 }
79 int rooted_lca(int r, int a, int b) {
80     return lca(a, b) ^ lca(a, r) ^ lca(b
81         , r); }
82 };
83 int main() {
84     ios::sync_with_stdio(false);
85     cin.tie(nullptr);
86
87     int n = 7;
88     HLD hld(n);
89
90     // build this tree:
91     //      0
92     //    /  \
93     //  1    2
94     // / \   / \
95     // 3  4 5
96     //    |
97     //    6
98     hld.add_edge(0, 1);
99     hld.add_edge(0, 2);
100    hld.add_edge(1, 3);
101    hld.add_edge(1, 4);
102    hld.add_edge(2, 5);
103    hld.add_edge(5, 6);
104
105    hld.build(0); // root at node 0
106
107    cout << "LCA(3,4) = " << hld.lca(3,4) <<
108        "\n"; // expected 1
109    cout << "LCA(3,6) = " << hld.lca(3,6) <<
110        "\n"; // expected 0
111    cout << "Distance(3,4) = " << hld.dist
112        (3,4) << "\n"; // expected 2
113    cout << "Distance(3,6) = " << hld.dist
114        (3,6) << "\n"; // expected 4
115    cout << "Is 0 ancestor of 6? " << (hld.
116        is_ancestor(0,6) ? "yes" : "no") <<
117        "\n"; // yes
118
119    return 0;
120 }

```

10.3 all longest path dfs

```

1 // all longest path (generalization of the
2 tree diameter problem)
3 vector<tuple<int, int, int>> dp{};
4 // [mx1, x, mx2] the path of mx1 goes
5 through x
6 int dfs1(int u, int p = -1) {
7     int mx = 0;
8     for (int v : adj[u]) {
9         if (v != p) {
10             int len = 1 + dfs1(v, u);
11             mx = max(mx, len);
12
13             auto& [mx1, x, mx2] = dp[u];
14             if (len >= mx1) {
15                 mx2 = mx1;

```

```

15         mx1 = len;
16         x = v;
17     }
18     else if (len > mx2) {
19         mx2 = len;
20     }
21 }
22 return mx;
23 }
24
25 void dfs2(int u, int p = -1) {
26     if (p != -1) {
27         int tmx;
28         { // calculate the longest path
29             through parent
30             auto& [mx1, x, mx2] = dp[p];
31             if (x != u) tmx = mx1 + 1;
32             else tmx = mx2 + 1;
33         }
34         { // update the path
35             auto& [mx1, x, mx2] = dp[u];
36             if (tmx >= mx1) {
37                 mx2 = mx1;
38                 mx1 = tmx;
39                 x = p;
40             }
41             else if (tmx > mx2) {
42                 mx2 = tmx;
43             }
44         }
45     }
46     for (int v : adj[u]) {
47         if (v != p) {
48             dfs2(v, u);
49         }
50     }
51 }
52
53 void all_longest_path() {
54     dp.resize(n);
55     dfs1(0), dfs2(0);
56 }

```

10.4 all longest path top sort

```

1 // all longest path in DAG
2 // 1. topological sort
3 vector<int> in(n, 0);
4 for (int i = 0; i < m; ++i) {
5     int a, b, w;
6     cin >> a >> b >> w;
7     adj[a].emplace_back(b, w);
8     in[b]++;
9 }
10
11 vector<int> topo; // sequence of top sort
12 queue<int> q;
13 for (int i = 0; i < n; ++i) {
14     if (in[i] == 0) {
15         q.push(i);
16     }
17 }

```

```

18 while (!q.empty()) {
19     int pa = q.front();
20     q.pop();
21     topo.push_back(pa);
22     for (auto& [child, w] : adj[pa]) {
23         in[child]--;
24         if (in[child] == 0) {
25             q.push(child);
26         }
27     }
28 }
29
30 // all longest path
31 vector<int> dist(n, INT_MIN);
32 vector<vector<int>> parents(n);
33 dist[0] = process[0];
34
35 for (int u : topo) {
36     for (auto& [v, w] : adj[u]) {
37         if (dist[v] < dist[u] + process[v] +
38             w) {
39             dist[v] = dist[u] + process[v] +
40                 w;
41             parents[v] = {u};
42         }
43         else if (dist[v] == dist[u] +
44             process[v] + w) {
45             parents[v].push_back(u);
46         }
47     }
48 }
49 cout << dist[n - 1];

```

10.5 Heavy-light decomposition (HLD)

```

1 const int maxn = 2e5;
2 int n, q;
3 int val[maxn];
4 int id[maxn];
5 int sz[maxn];
6 int parent[maxn];
7 int depth[maxn];
8 int tp[maxn];
9 vector<int> adj[maxn];
10 int timer = 0;
11
12 int dfs_sz(int u, int p) {
13     sz[u] = 1;
14     parent[u] = p;
15
16     // calculate size of each subtree
17     for (int v : adj[u]) {
18         if (v != p) {
19             depth[v] = depth[u] + 1;
20             sz[u] += dfs_sz(v, u);
21         }
22     }
23     return sz[u];
24 }
25

```



```

26 void dfs_hld(int u, int p, int top, SGT<int,
    MergeMax>& tree) {
27     id[u] = timer++;
28     tp[u] = top;
29     tree.update(id[u], val[u]);
30
31     // find the heavy child
32     int h_v = -1, h_sz = -1;
33     for (int v : adj[u]) {
34         if (v != p) {
35             if (h_sz < sz[v]) {
36                 h_sz = sz[v];
37                 h_v = v;
38             }
39         }
40     }
41     if (h_v == -1) {
42         return;
43     }
44
45     // continue the chain
46     dfs_hld(h_v, u, top, tree);
47     // start a new chain for each light
48     // child
49     for (int v : adj[u]) {
50         if (v != p && v != h_v) {
51             dfs_hld(v, u, v, tree);
52         }
53     }
54 }

```

```

55 int path(int x, int y, SGT<int, MergeMax>&
    tree) {
56     int ans = 0;
57     // move x and y to their top nodes until
58     // they are in the same chain
59     while (tp[x] != tp[y]) {
60         if (depth[tp[x]] < depth[tp[y]])
61             swap(x, y);
62         ans = max(ans, tree.query(id[tp[x]],
63             id[x] + 1));
64         x = parent[tp[x]];
65     }
66     if (depth[x] > depth[y]) swap(x, y);
67     ans = max(ans, tree.query(id[x], id[y] +
68         1));
69     return ans;
70 }
71
72 void solve() {
73     cin >> n >> q;
74     SGT<int, MergeMax> tree(n, -1);
75
76     for (int i = 0; i < n; ++i) {
77         cin >> val[i];
78     }
79     for (int i = 0; i < n - 1; ++i) {
80         int a, b;
81         cin >> a >> b;
82         a--, b--;
83         adj[a].push_back(b);
84         adj[b].push_back(a);
85     }
86
87     dfs_sz(0, 0);
88     dfs_hld(0, 0, 0, tree);

```

```

85     cerr << "tp: ";
86     for (int i = 0; i < n; ++i) {
87         cerr << tp[i] << " ";
88     }
89     cerr << "\n";
90
91     while (q--) {
92         int mode;
93         cin >> mode;
94         if (mode == 1) {
95             int s, x;
96             cin >> s >> x;
97             s--;
98             val[s] = x;
99             tree.update(id[s], val[s]);
100         }
101         else {
102             int a, b;
103             cin >> a >> b;
104             cout << path(a - 1, b - 1, tree)
105                 << " ";
106         }
107     }
108 }

```

10.6 get kth ancestor

```

1 void dfs(int u, int p = -1) {
2     up[0][u] = (p == -1 ? u : p);
3     for (int v : adj[u]) {
4         if (v != p) {
5             dfs(v, u);
6         }
7     }
8 }
9
10 int get_kth_ancestor(int x, int k) {
11     for (int i = 0; i <= LOG; ++i) {
12         if (k & (1LL << i)) {
13             x = up[i][x];
14         }
15     }
16     return (x == 0 ? -1 : x);
17 }
18
19 void solve() {
20     int n, q;
21     cin >> n >> q;
22
23     adj[0].push_back(1);
24     for (int i = 2; i <= n; ++i) {
25         int u;
26         cin >> u;
27         adj[u].push_back(i);
28     }
29
30     dfs(0, -1);
31
32     for (int k = 1; k <= LOG; ++k) {
33         for (int v = 1; v <= n; ++v) {
34             up[k][v] = up[k - 1][up[k - 1][v]
35                 ];
36         }
37     }
38 }

```

```

35     }
36 }
37
38 while (q--) {
39     int x, k;
40     cin >> x >> k;
41     cout << get_kth_ancestor(x, k) << "\n";
42 }
43 }

```

10.7 path maximum edge between u, v in tree

```

1 int depth[maxn];
2 int up[LOG][maxn];
3 int maxEdge[LOG][maxn];
4
5 void dfs(int u, int p = -1, int w = 0) {
6     up[0][u] = (p == -1 ? u : p);
7     maxEdge[0][u] = (p == -1 ? 0 : w);
8     for (auto &[v, rw] : adj[u]) {
9         if (v != p) {
10             depth[v] = depth[u] + 1;
11             dfs(v, u, rw);
12         }
13     }
14 }
15
16 int lca_weight(int a, int b) {
17     if (depth[a] < depth[b]) swap(a, b);
18     int diff = depth[a] - depth[b];
19     int ans = 0;
20
21     for (int k = 0; diff; ++k) {
22         if (diff & 1) {
23             ans = max(ans, maxEdge[k][a]);
24             a = up[k][a];
25         }
26         diff >>= 1;
27     }
28     if (a == b) {
29         return ans;
30     }
31     for (int k = LOG - 1; k >= 0; --k) {
32         if (up[k][a] != up[k][b]) {
33             ans = max(ans, maxEdge[k][a]);
34             ans = max(ans, maxEdge[k][b]);
35             a = up[k][a];
36             b = up[k][b];
37         }
38     }
39
40     ans = max(ans, maxEdge[0][a]);
41     ans = max(ans, maxEdge[0][b]);
42     return ans;
43 }
44
45 void solve() {
46     dfs(0, -1, 0);
47     for (int k = 1; k < LOG; ++k) {
48         for (int v = 0; v < n; ++v) {

```

```

49             up[k][v] = up[k - 1][up[k - 1][v]
50                 ];
51             maxEdge[k][v] = max(maxEdge[k -
52                 1][v], maxEdge[k - 1][up[k -
53                 1][v]]);
54         }
55     }
56     int path_max = lca_weight(0, 1);
57 }

```

10.8 binary jumping

```

1 const int LOG = 32;
2 const int maxn = 2e5 + 2;
3
4 int depth[maxn];
5 int up[LOG][maxn];
6 vector<int> adj[maxn];
7
8 void dfs(int u, int p = -1) {
9     up[0][u] = (p == -1 ? u : p);
10    for (int v : adj[u]) {
11        if (v != p) {
12            depth[v] = depth[u] + 1;
13            dfs(v, u);
14        }
15    }
16 }
17
18 void solve() {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21
22     int n, q;
23     cin >> n >> q;
24
25     for (int i = 2; i <= n; ++i) {
26         int u;
27         cin >> u;
28         adj[u].push_back(i);
29     }
30
31     dfs(1, -1); // root = 1
32
33     for (int k = 1; k < LOG; ++k) {
34         for (int v = 1; v <= n; ++v) {
35             up[k][v] = up[k - 1][up[k - 1][v]
36                 ];
37         }
38     }
39
40     while (q--) {
41         int a, b;
42         cin >> a >> b;
43         cout << lca(a, b) << "\n";
44     }
45 }

```

10.9 tree diameter

```

1 int diam = 0;
2
3 int dfs(int u, int p = -1) {
4     int mx = 0;
5     for (int v : adj[u]) {
6         if (v != p) {
7             int len = 1 + dfs(v, u);
8             diam = max(diam, mx + len);
9             mx = max(mx, len);
10        }
11    }
12    return mx;
13 }

```

10.10 all longest path

```

1 int fir[maxn]; // Length of the longest
// downward path from u into its subtree.
2 int sec[maxn]; // Length of the second
// longest downward path from u
3 int res[maxn];
4
5 void dfs1(int u, int p) {
6     for (int v : adj[u]) {
7         if (v != p) {
8             dfs1(v, u);
9             if (fir[v] + 1 > fir[u]) {
10                sec[u] = fir[u];
11                fir[u] = fir[v] + 1;
12            }
13            else if (fir[v] + 1 > sec[u]) {
14                sec[u] = fir[v] + 1;
15            }
16        }
17    }
18 }
19
20 // to_p: the best path length that comes
// from the parent's side
21 void dfs2(int u, int p, int to_p) {
22     res[u] = max(to_p, fir[u]);
23
24     for (int v : adj[u]) {
25         if (v != p) {
26             if (fir[v] + 1 == fir[u]) {
27                 dfs2(v, u, max(to_p, sec[u])
28                     + 1);
29             }
30             else {
31                 dfs2(v, u, res[u] + 1);
32             }
33         }
34     }
35 }
36
37 // usage
38 dfs1(1, 0);
39 dfs2(1, 0, 0);
40
41 // Now res[i] is the maximum distance from
// node i to any other node

```

```

40 for (int i = 1; i <= n; i++) {
41     cout << res[i] << " ";
42 }

```

10.11 tree diameter (len,end)

```

1 array<int, 2> dfs(int u, int w = -1) {
2     array<int, 2> mx{0, u}; // {length,
// farthest leaf}
3     for (auto& v : adj[u]) {
4         if (v == w) continue;
5         auto [len, leaf]{dfs(v, u)};
6         mx = max(mx, {len + 1, leaf});
7     }
8     return mx;
9 }
10
11 array<int, 3> tree_diameter(int a = 0) {
12     auto b{dfs(a)[1]}; // farthest node
// from 'a'
13     auto [l, c]{dfs(b)}; // farthest node
// from 'b'
14     return {l, b, c}; // {diameter
// length, endpoint1, endpoint2}
15 }

```

10.12 minimum distance in functional graph

```

1 // Given a functional graph (each node has
// exactly one outgoing edge), answer
2 // queries about the minimum distance
// between two nodes, or determine that
3 // it's impossible to get from one to the
// other.
4 // There are three cases:
5 // 1. Both in Tree
6 // 2. Both in Cycle
7 // 3. One in Tree, one in Cycle
8
9 int main() {
10     int planet_num;
11     int query_num;
12     std::cin >> planet_num >> query_num;
13     vector<int> next(planet_num);
14     vector<vector<int>> before(planet_num);
15     for (int p = 0; p < planet_num; p++) {
16         std::cin >> next[p];
17         next[p]--;
18         before[next[p]].push_back(p);
19     }
20
21     /*
22     * -2 = We haven't even got to processing
// this planet yet.
23     * -1 = This node is part of a tree.
24     * >= 0: the ID of the cycle the planet
// belongs to.
25     */

```

```

26 vector<int> cycle_id(planet_num, -2);
27 // Each map, given a planet #, returns the
// index of that planet in the
28 // cycle.
29 vector<std::map<int, int>> cycles;
30 for (int p = 0; p < planet_num; p++) {
31     if (cycle_id[p] != -2) { continue; }
32     vector<int> path{p};
33     int at = p;
34     while (cycle_id[next[at]] == -2) {
35         at = next[at];
36         cycle_id[at] = -3; // Leave
// breadcrumbs for this iteration
37     }
38     path.push_back(at);
39
40     std::map<int, int> cycle;
41     bool in_cycle = false;
42     for (int i : path) {
43         in_cycle = in_cycle || i == next[at];
44         if (in_cycle) { cycle[i] = cycle.size()
45             (); }
46         cycle_id[i] = in_cycle ? cycles.size()
47             : -1;
48     }
49     cycles.push_back(cycle);
50 }
51
52 /*
53 * Precalculate the distance from each
// planet to its cycle with BFS.
54 * (cyc_dist[p] = 0) if p is part of a
// cycle.
55 */
56 vector<int> cyc_dist(planet_num);
57 for (int p = 0; p < planet_num; p++) {
58     // Check if this planet is part of a
// cycle.
59     if (cycle_id[next[p]] == -1 || cycle_id[
60         p] != -1) { continue; }
61     cyc_dist[p] = 1;
62     vector<int> stack(before[p]);
63     while (!stack.empty()) {
64         int curr = stack.back();
65         stack.pop_back();
66         cyc_dist[curr] = cyc_dist[next[curr]]
67             + 1;
68         stack.insert(stack.end(), before[curr]
69             .begin(), before[curr].end());
70     }
71 }
72
73 // Initialize the binary jumping arrays.
74 int log2 = std::ceil(std::log2(planet_num)
75 );
76 vector<vector<int>> pow2_ends(planet_num,
77     vector<int>(log2 + 1));
78 for (int p = 0; p < planet_num; p++) {
79     pow2_ends[p][0] = next[p];
80 }
81 for (int i = 1; i <= log2; i++) {
82     for (int p = 0; p < planet_num; p++) {
83         pow2_ends[p][i] = pow2_ends[pow2_ends[
84             p][i - 1]][i - 1];
85     }
86 }
87
88 /*
89 * Given a starting planet & dist, returns
// the planet we end up at
90 * if we use the teleporter dist times.
91 */
92 auto advance = [&](int pos, int dist) {
93     for (int pow = log2; pow >= 0; pow--) {
94         if ((dist & (1 << pow)) != 0) { pos =
95             pow2_ends[pos][pow]; }
96     }
97     return pos;
98 }
99
100 for (int q = 0; q < query_num; q++) {
101     int u, v; // going from u to v
102     std::cin >> u >> v;
103     u--;
104     v--;
105     if (cycle_id[pow2_ends[u][log2]] !=
106         cycle_id[pow2_ends[v][log2]]) {
107         cout << -1 << '\n';
108         continue;
109     }
110     if (cycle_id[u] != -1 || cycle_id[v] !=
111         -1) {
112         if (cycle_id[v] == -1 && cycle_id[u]
113             != -1) {
114             cout << -1 << '\n';
115             continue;
116         }
117         // Handle the 2nd & 3rd cases at the
118         // same time.
119         int dist = cyc_dist[u];
120         int u_cyc = advance(u, cyc_dist[u]);
121         // The root of u's tree
122
123         std::map<int, int> &cyc = cycles[
124             cycle_id[u_cyc]]; // u and v's
// cycle
125         int u_ind = cyc[u_cyc];
126         int v_ind = cyc[v];
127         int diff = u_ind <= v_ind ? v_ind -
128             u_ind : cyc.size() - (u_ind -
129             v_ind);
130         cout << dist + diff << '\n';
131     } else {
132         if (cyc_dist[v] > cyc_dist[u]) {
133             cout << -1 << '\n';
134             continue;
135         }
136         int diff = cyc_dist[u] - cyc_dist[v];
137         cout << (advance(u, diff) == v ? diff
138             : -1) << '\n';
139     }
140 }

```

10.13 union length of two tree paths

```

1 /*
2 Problem: Given a tree and queries (a,b,c,d),
// find the size of the union of nodes on
// paths a+b and c+d.
3

```

```

4 Let P1 be the nodes on the path a↔b and P2
  the nodes on c↔d.
5 We want |P1 ∩ P2| = |P1| + |P2| - |P1 ∪ P2|.
6 |P1| = dist(a,b) + 1, |P2| = dist(c,d) + 1.
  So the only hard part is |P1 ∩ P2|.
7
8 Key facts for trees:
9
10 The intersection of two simple paths is
  itself either empty or a simple path.
11
12 Any endpoint of that intersection must be
  one of the LCAs among endpoint pairs of
  the two paths.
13
14 So for each query:
15
16 compute the 6 candidate LCAs,
17
18 keep those candidates that lie on both paths
  (test with distances),
19
20 if none → intersection size = 0; if one →
  intersection size = 1;
21 otherwise take the two kept candidates with
  greatest depth (they are the endpoints
  of the intersection)
22 and intersection size = dist(p,q) + 1.
23 */
24
25 const int LOG = 32;
26
27 const int maxn = 1e5;
28 int depth[maxn];
29 vector<int> adj[maxn];
30 int up[LOG + 1][maxn];
31 int n, q;
32
33 void dfs(int u, int p = -1) {
34     up[0][u] = (p == -1 ? u : p);
35     for (int v : adj[u]) {
36         if (v != p) {
37             depth[v] = depth[u] + 1;
38             dfs(v, u);
39         }
40     }
41 }
42
43 int dist(int a, int b) {
44     int c = lca(a, b);
45     return depth[a] + depth[b] - 2 * depth[c];
46 }
47
48 // check z on path u-v
49 bool on_path(int z, int u, int v) {
50     return dist(u, z) + dist(z, v) == dist(u, v);
51 }
52
53 void solve() {
54     cin >> n >> q;
55     for (int i = 0; i < n - 1; ++i) {
56         int a, b;
57         cin >> a >> b;

```

```

59     a--, b--;
60     adj[a].push_back(b);
61     adj[b].push_back(a);
62 }
63 memset(depth, 0, sizeof(depth));
64 dfs(0, -1);
65
66 for (int k = 1; k < LOG; ++k) {
67     for (int v = 0; v < n; ++v) {
68         up[k][v] = up[k - 1][up[k - 1][v]];
69     }
70 }
71
72 while (q--) {
73     int a, b, c, d;
74     cin >> a >> b >> c >> d;
75     a--, b--, c--, d--;
76     int len1 = dist(a, b) + 1;
77     int len2 = dist(c, d) + 1;
78
79     array<int, 6> cand = {
80         lca(a, b), lca(c, d),
81         lca(a, c), lca(a, d),
82         lca(b, c), lca(b, d)
83     };
84
85     vector<int> good;
86     for (int z : cand) {
87         if (on_path(z, a, b) && on_path(
88             z, c, d)) {
89             good.push_back(z);
90         }
91     }
92
93     int inter = 0;
94     if (good.size() == 1) {
95         inter = 1;
96     }
97     else if (good.size() > 1) {
98         sort(good.begin(), good.end(),
99             [&](int x, int y) {
100                 return depth[x] > depth[y];
101             });
102         int p = good[0], qn = good[1];
103         inter = dist(p, qn) + 1;
104     }
105     cout << (len1 + len2 - inter) << "\n";
106 }

```

10.14 Centroid decomposition

```

1 // Centroid Decomposition
2 // Problem: Given a tree with N nodes (1-
  indexed) and M queries. Each query is
  one of the following two types:
3 // 1 v: Paint node v red.
4 // 2 v: Print the minimal distance between
  node v and any red node.
5 // Initially, only node 1 is red.
6 // Constraints: 1 ≤ N, M ≤ 10^5

```

```

7 // Time complexity: O((N + M) Log N)
8
9 vector<vector<int>> adj;
10 vector<int> subtree_size;
11 // min_dist[v] = the minimal distance
  between v and a red node
12 vector<int> min_dist;
13 vector<bool> is_removed;
14 vector<vector<pii>> ancestors;
15
16 int get_subtree_size(int u, int p = -1) {
17     subtree_size[u] = 1;
18     for (int v : adj[u]) {
19         if (v == p || is_removed[v])
20             continue;
21         subtree_size[u] += get_subtree_size(
22             v, u);
23     }
24     return subtree_size[u];
25
26 int get_centroid(int u, int tree_size, int p
  = -1) {
27     for (int v : adj[u]) {
28         if (v == p || is_removed[v])
29             continue;
30         if (subtree_size[v] * 2 > tree_size)
31             return get_centroid(v, tree_size
32                 , u);
33     }
34     return u;
35 }
36
37 // Calculate the distance between current `u`
  and the `centroid` it belongs.
38 void get_dist(int u, int centroid, int p =
  -1, int cur_dist = 1) {
39     for (int v : adj[u]) {
40         if (v == p || is_removed[v])
41             continue;
42         cur_dist++;
43         get_dist(v, centroid, u, cur_dist);
44         cur_dist--;
45     }
46     ancestors[u].emplace_back(centroid,
47         cur_dist);
48 }
49
50 void build_centroid_decomp(int u = 0) {
51     int centroid = get_centroid(u,
52         get_subtree_size(u));
53
54     // For all nodes in the subtree rooted
55     at `centroid`, calculate their
56     distance to the centroid
57     for (int v : adj[centroid]) {
58         if (is_removed[v]) continue;
59         get_dist(v, centroid, centroid);
60     }
61
62     is_removed[centroid] = true;
63     for (int v : adj[centroid]) {
64         if (is_removed[v]) continue;
65         build_centroid_decomp(v);

```

```

59     }
60 }
61
62 // Paint `node` red by updating all of its
  ancestors' minimal distances to a red
  node
63 void paint(int u) {
64     for (auto &[ancestor, dist] : ancestors[
65         u]) {
66         min_dist[ancestor] = min(min_dist[
67             ancestor], dist);
68     }
69     min_dist[u] = 0;
70 }
71
72 // Print the minimal distance between `u` to
  a red node.
73 void query(int u) {
74     int ans = min_dist[u];
75     for (auto &[ancestor, dist] : ancestors[
76         u]) {
77         if (!dist) continue;
78         ans = min(ans, dist + min_dist[
79             ancestor]);
80     }
81     cout << ans << "\n";
82 }
83
84 void solve() {
85     int N, M;
86     cin >> N >> M;
87
88     adj.assign(N, vector<int>());
89     for (int i = 0; i < N - 1; ++i) {
90         int a, b;
91         cin >> a >> b;
92         a--, b--;
93         adj[a].push_back(b);
94         adj[b].push_back(a);
95     }
96
97     subtree_size.assign(N, 0);
98     ancestors.assign(N, vector<pii>());
99     is_removed.assign(N, false);
100     build_centroid_decomp();
101
102     min_dist.assign(N, INF);
103     paint(0);
104     for (int i = 0; i < M; ++i) {
105         int t, v;
106         cin >> t >> v;
107         v--;
108         if (t == 1) {
109             paint(v);
110         }
111         else {
112             query(v);
113         }
114     }

```

11 default

11.1 debug

```
1 #ifndef DEBUG
2 #define dbg(...) {\
3     fprintf(stderr, "%s - %d : (%s) = ",
4         __PRETTY_FUNCTION__, __LINE__, #
5         __VA_ARGS__); \
6     _DO(__VA_ARGS__); \
7 }
8 template<typename I> void _DO(I&&x){cerr<<x
9     <<endl;}
10 template<typename I, typename...T> void _DO(I
11     &&x, T&&...tail){cerr<<x<<" "; _DO(tail
12     ...);}
13 #else
14 #define dbg(...)
15 #endif
```

11.2 template

```
1 // alias g++='g++ -std=c++14 -fsanitize=
2     undefined -Wall -Wextra -Wshadow -D
3     LOCAL'
4
5 #include <bits/stdc++.h>
6 using namespace std;
7
8 #ifndef LOCAL
9 #include "../debug.h"
10 #else
11 #pragma GCC optimize("O3,unroll-loops")
12 #pragma GCC target("avx2,bmi,bmi2,lzcnt,
13     popcnt")
14 #define debug(...) ((void)0)
15 #define orange(...) ((void)0)
16 #endif
17
18 #define int long long
19 #define SPEEDY ios_base::sync_with_stdio(
20     false); cin.tie(0); cout.tie(0);
21
22 void solve() {
23
24 }
25
26 signed main() {
27     SPEEDY;
28     return 0;
29 }
```

11.3 vimrc

```
1 set nu
2 set ts=4
3 set shiftwidth=4
```

```
4 set autoindent smartindent
5 set cindent
6
7 imap jk <ESC>
8 inoremap {<CR> {<CR><ESC>ko
```

12 other

12.1 Nim game

```
1 | a1^a2^a3^...^an != 0 ? A win : B win
```

12.2 找小於 n 所有出現的 1 數量

```
1 current == 0 higher * factor
2 current == 1 higher * factor + lower + 1
3 other current (higher + 1) * factor
```

13 other language

13.1 python heap

```
1 import heapq
2
3 heap = [7,1,2,2]
4 heapq.heapify(heap)
5 print(heap) # [1, 2, 2, 7]
6 heapq.heappush(heap, 5)
7 print(heap) # [1, 2, 2, 7, 5]
8 print(heapq.heappop(heap)) # 1
9 print(heap) # [2, 2, 5, 7]
```

13.2 java

13.2.1 文件操作

```
1 import java.io.*;
2 import java.util.*;
3 import java.math.*;
4 import java.text.*;
5
6 public class Main{
7     public static void main(String args[]){
8         throws FileNotFoundException,
9             IOException
10         Scanner sc = new Scanner(new FileReader(
11             "a.in"));
12         PrintWriter pw = new PrintWriter(new
13             FileWriter("a.out"));
```

```
10 int n,m;
11 n=sc.nextInt();
12 m=sc.nextInt();
13
14 for(ci=1; ci<=c; ++ci){
15     pw.println("Case #"+ci+": easy for
16         output");
17 }
18
19 pw.close();
20 sc.close();
21 }
```

13.2.2 优先队列

```
1 PriorityQueue queue = new PriorityQueue( 1,
2     new Comparator(){
3     public int compare( Point a, Point b ){
4         if( a.x < b.x || a.x == b.x && a.y < b.y )
5             return -1;
6         else if( a.x == b.x && a.y == b.y )
7             return 0;
8         else return 1;
9     }
10 });
```

13.2.3 Map

```
1 Map map = new HashMap();
2 map.put("sa", "dd");
3 String str = map.get("sa").toString();
4
5 for(Object obj : map.keySet()){
6     Object value = map.get(obj );
7 }
```

13.2.4 sort

```
1 static class cmp implements Comparator{
2     public int compare(Object o1, Object o2){
3         BigInteger b1=(BigInteger)o1;
4         BigInteger b2=(BigInteger)o2;
5         return b1.compareTo(b2);
6     }
7 }
8 public static void main(String[] args)
9     throws IOException{
10     Scanner cin = new Scanner(System.in);
11     int n;
12     n=cin.nextInt();
13     BigInteger[] seg = new BigInteger[n];
14     for (int i=0;i<n;i++)
15         seg[i]=cin.nextBigInteger();
16     Arrays.sort(seg, new cmp());
```

13.3 python output

```
1 hello = 'Hello'
2 world = 7122
3 print(f'hello {world}') # Hello 7122
4
5 import math
6 print(f'PI is approximately {math.pi:.3f}.')
7 # PI is approximately 3.142.
8
9 print('AAA {} BBB {}'.format('Jin', 'Kela'
10     ''))
11 # AAA Jin BBB "KeLa!"
12
13 hello = 'hello, world\n'
14 hellos = repr(hello)
15 print(hellos) # 'hello, world\n'
16
17 x = 32.5
18 y = 40000
19 print(repr((x, y, ('spam', 'eggs'))))
20 # "(32.5, 40000, ('spam', 'eggs'))"
21
22 x = 7
23 print(eval('3 * x')) # 21
```

13.4 python 大數因數分解

```
1 # 大數因數分解 (使用 Pollard's Rho 與 Miller
2     -Rabin)
3 import sys, random
4 from math import gcd
5
6 # Miller-Rabin 檢定 (機率性質數判定)
7 def is_probable_prime(n, k=12):
8     if n < 2:
9         return False
10     # 先檢查一些小質數
11     small_primes =
12         [2,3,5,7,11,13,17,19,23,29]
13     for p in small_primes:
14         if n % p == 0:
15             return n == p
16     # 把 n-1 寫成 d * 2^s
17     d = n - 1
18     s = 0
19     while d % 2 == 0:
20         d //= 2
21         s += 1
22     # 重複 k 次隨機測試
23     for _ in range(k):
24         a = random.randrange(2, n - 1) # 隨
25             機挑一個測試基數
26         x = pow(a, d, n)
27         if x == 1 or x == n - 1:
28             continue
29         composite = True
30         for __ in range(s - 1):
31             x = pow(x, 2, n)
32             if x == n - 1:
33                 composite = False
```

```

31         break
32     if composite:
33         return False
34     return True
35
36 # Pollard's Rho 演算法 (找非平凡因數)
37 def pollards_rho(n):
38     if n % 2 == 0:
39         return 2
40     if n % 3 == 0:
41         return 3
42 # 隨機多項式 (x^2 + c) mod n
43 while True:
44     c = random.randrange(1, n-1) # 隨機挑選常數 c
45     x = random.randrange(2, n-1) # 起始點
46     y = x
47     d = 1
48     while d == 1:
49         x = (pow(x, 2, n) + c) % n # x -> f(x)
50         y = (pow(y, 2, n) + c) % n # y -> f(y) · 走兩步
51         y = (pow(y, 2, n) + c) % n
52         d = gcd(abs(x - y), n) # 計算兩者差的 gcd
53     if d == n: # 失敗就重試
54         break
55     if d > 1 and d < n: # 找到非平凡因數
56         return d
57
58 # 遞迴分解
59 def factor(n, out):
60     if n == 1:
61         return
62     if is_probable_prime(n):
63         out.append(n)
64     else:
65         d = pollards_rho(n)
66         while d is None or d == n: # 偶爾失敗就重試
67             d = pollards_rho(n)
68         factor(d, out)
69         factor(n // d, out)
70
71 def main():
72     data = sys.stdin.read().strip().split()
73     if not data:
74         return
75     # 每個 token 當作一個數字
76     for token in data:
77         try:
78             n = int(token)
79         except:
80             continue
81     if n <= 1:
82         print(n)
83         continue
84     facs = []
85     factor(n, facs)
86     facs.sort()

```

```

87     # 輸出因數
88     print(" ".join(str(x) for x in facs))
89
90 if __name__ == "__main__":
91     random.seed() # 使用系統時間作為隨機種子
92     main()

```

13.5 decimal

```

1 # 使用 decimal 模組來處理高精度小數運算
2 from decimal import *
3 setcontext(Context(prec=MAX_PREC, Emax=MAX_EMAX, rounding=ROUND_FLOOR))
4 print(Decimal(input()) * Decimal(input()))
5
6 # 將小數轉成分數 · 方便做近似或理論分析 · 且可以限制分母大小。
7 from fractions import Fraction
8 Fraction('3.14159').limit_denominator(10).numerator # 22
9
10 # 設定精確度
11 from decimal import Decimal, getcontext
12
13 # 精確位數設定
14 getcontext().prec = 70
15
16 n = 100
17 # 指定 n 為高精確度的物件
18 n = Decimal(n)
19 n /= 7
20 print(n)
21
22 # 將小數轉成分數
23 from fractions import Fraction
24
25 n = 1.5654
26 # 建立一個轉換物件
27 n = Fraction(n)
28
29 print(n)

```

13.6 python 大數排序

```

1 # 大數排序
2 # Line one n : 多少數字
3 # next line : 依序輸入每行一個
4 # sort : sort + lambda
5 from sys import stdin
6
7 data = stdin.read().splitlines()
8
9 i = 0
10
11 while(i < len(data)):
12     T = int(data[i].strip())

```

```

13     i += 1
14     temp = [int(data[i + index].strip()) for index in range(T)]
15     i += T
16     temp = sorted(temp, key = lambda x : (x))
17     for ele in temp:
18         print(ele)

```

13.7 python 大數計算 2

```

1 # 單行輸入
2 # format : n1, operation, n2
3 from sys import stdin
4
5 data = stdin.read().splitlines()
6
7 limit = len(data)
8 i = 0
9
10 while(i < limit):
11     a, operation, b = map(str, data[i].split())
12     a, b = int(a), int(b)
13     i += 1
14     if(operation == '+'):
15         print(int(a + b))
16     elif(operation == '-'):
17         print(int(a - b))
18     elif(operation == '*'):
19         print(int(a * b))
20     else:
21         print(int(a // b))

```

13.8 python input

```

1 ans = sum(map(float, input().split()))
2 # input: 1.1 2.2 3.3 4.4 5.5
3 print(ans) # 16.5
4
5 (n, m) = map(int, input().split()) # 300 200
6 print(n * m) # 60000
7
8 Arr = list(map(int, input().split()))
9 # input: 1 2 3 4 5
10 print(Arr) # [1, 2, 3, 4, 5]

```

13.9 python 大數計算

```

1 # 讀取多行輸入
2 # line one first number
3 # line two operation
4 # line three second number
5 from sys import stdin
6
7 data = stdin.read().splitlines()

```

```

8 limit = len(data)
9 i = 0
10 while(i < limit):
11     a = int(data[i].strip())
12     i += 1
13     operation = data[i].strip()
14     i += 1
15     b = int(data[i].strip())
16     i += 1
17     if(operation == '*'):
18         print(int(a * b))
19     else:
20         print(int(a / b))

```

13.10 base-6 pow

```

1 import sys
2
3 sys.setrecursionlimit(2000000)
4
5 def solve():
6     S = sys.stdin.readline().strip()
7     try:
8         N = int(S)
9     except ValueError:
10         N = 0
11     if N == 0:
12         print(1)
13         return
14     low = 1
15     high = 700000 # Safely Larger than 500000 * 1.3
16     ans = high + 1
17     while low <= high:
18         L_mid = (low + high) // 2
19         if L_mid <= 0: # Avoid 6^0 or 6^negative
20             low = L_mid + 1
21             continue
22         val = pow(6, L_mid)
23         if val > N:
24             ans = L_mid
25             high = L_mid - 1
26         else:
27             low = L_mid + 1
28     print(ans)
29 solve()

```

14 zformula

14.1 formula

14.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形 · 面積 = 內部格點數 + 邊上格點數/2 - 1

14.1.2 圖論

1. 對於平面圖 · $F = E - V + C + 1$ · C 是連通分量數
2. 對於平面圖 · $E \leq 3V - 6$
3. 對於連通圖 G · 最大獨立點集的大小設為 $I(G)$ · 最大匹配大小設為 $M(G)$ · 最小點覆蓋設為 $Cv(G)$ · 最小邊覆蓋設為 $Ce(G)$ · 對於任意連通圖：

- (a) $I(G) + Cv(G) = |V|$
(b) $M(G) + Ce(G) = |V|$
4. 對於連通二分圖：
- (a) $I(G) = Cv(G)$
(b) $M(G) = Ce(G)$

14.1.3 學長公式

1. $\sum_{d|n} \phi(n) = n$
2. Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
3. $\gamma = 0.57721566490153286060651209008240243104215$
4. 選轉矩陣 $M(\theta) = \left(\begin{array}{cc} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{array} \right)$

14.1.4 基本數論

1. $\sum_{d|n} \mu(n) = [n == 1]$
2. $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
3. $\sum_{i=1}^n \sum_{j=1}^m \text{互質數量} = \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
4. $\sum_{i=1}^n \sum_{j=1}^n lcm(i, j) = n \sum_{d|n} d \times \phi(d)$

14.1.5 排組公式

1. k 卡特蘭 $\frac{C_n^{kn}}{n(k-1)+1} \cdot C_m^n = \frac{n!}{m!(n-m)!}$
2. $H(n, m) \cong x_1 + x_2 \dots + x_n = k, num = C_k^{n+k-1}$
3. Stirling number of 2^{nd} , n 人分 k 組方法數目
- (a) $S(0, 0) = S(n, n) = 1$
(b) $S(n, 0) = 0$
(c) $S(n, k) = kS(n - 1, k) + S(n - 1, k - 1)$
4. Bell number, n 人分任意多組方法數目
- (a) $B_0 = 1$
(b) $B_n = \sum_{i=0}^n S(n, i)$
(c) $B_{n+1} = \sum_{k=0}^n C_k^n B_k$
(d) $B_{p+n} \equiv B_n + B_{n+1} \text{ mod } p$, p is prime
(e) $B_{p^m+n} \equiv mB_n + B_{n+1} \text{ mod } p$, p is prime
(f) From $B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$
5. Derangement, 錯排, 沒有人在自己位置上
- (a) $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \dots + (-1)^n \frac{1}{n!})$
(b) $D_n = (n - 1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
(c) From $D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$

6. Binomial Equality

- (a) $\sum_k \binom{r}{m+k} \binom{s}{n-k} = \binom{r+s}{m+n}$
(b) $\sum_k \binom{l}{m+k} \binom{s}{n+k} = \binom{l+s}{l-m+n}$
(c) $\sum_k \binom{l}{m+k} \binom{s+k}{n} (-1)^k = (-1)^{l+m} \binom{s-m}{n-l}$
(d) $\sum_{k \leq l} \binom{l-k}{m} \binom{s}{n-k} (-1)^k = \frac{(-1)^{l+m} \binom{s-m-1}{l-n-m}}{(-1)^{l+m} \binom{s-m-1}{l-n-m}}$
(e) $\sum_{0 \leq k \leq l} \binom{l-k}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
(f) $\binom{r}{k} = (-1)^k \binom{k}{k-r-1}$
(g) $\binom{r}{m} \binom{m}{k} = \binom{r}{k} \binom{r-k}{m-k}$
(h) $\sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{m+1}$
(i) $\sum_{0 \leq k \leq n} \binom{k}{m} = \binom{n+1}{m+1}$
(j) $\sum_{k \leq m} \binom{m+r}{k} x^k y^{m-k} = \sum_{k \leq m} \binom{-r}{k} (-x)^k (x+y)^{m-k}$

14.1.6 冪次, 冪次和

1. $a^{b \% p} P = a^{b \% \varphi(p) + \varphi(p)} , b \geq \varphi(p)$
2. $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
3. $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
4. $1^5 + 2^5 + 3^5 + \dots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
5. $0^k + 1^k + 2^k + \dots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n + 1$
6. $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^n C_k^{n+1} B_k m^{n+1-k}$
7. $\sum_{j=0}^m C_j^{m+1} B_j = 0, B_0 = 1$
8. 除了 $B_1 = -1/2$ · 剩下的奇數項都是 0
9. $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

14.1.7 循環小數轉分數

1. 若 $x = 0.\overline{a}$ · 則

$$x = \frac{a}{\underbrace{99 \dots 9}_{k \text{ digits}}}$$

- 其中 a 為循環節 · k 為循環節的位數 ·
2. 例子：

$$0.\overline{37} = \frac{37}{99}$$

$$0.\overline{5} = \frac{5}{9}$$

3. 純循環小數：若 $x = 0.\overline{a}$ · 其中 a 為循環節 · 長度為 k ·

$$x = \frac{a}{\underbrace{99 \dots 9}_{k \text{ digits}}}$$

例：

$$0.\overline{37} = \frac{37}{99}, \quad 0.\overline{5} = \frac{5}{9}$$

4. 混循環小數：若 $x = 0.b\overline{a}$ · 其中 b 為前綴 · 長度 m · a 為循環節 · 長度 k ·

$$x = \frac{(b \cdot 10^k + a) - b}{10^m(10^k - 1)}$$

例：

$$0.12\overline{3} = \frac{(12 \cdot 10^1 + 3) - 12}{10^2(10^1 - 1)} = \frac{123 - 12}{100 \cdot 9} = \frac{111}{900} = \frac{37}{300}$$

14.1.8 常見級數與組合公式

1. 平方和公式：

$$1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

$$4^2 + 6^2 + \dots + (2n)^2 = \frac{(2n)(n+1)(2n+1)}{3}$$

$$1^2 + 3^2 + \dots + (2n+1)^2 = \frac{n(2n-1)(2n+1)}{3}$$

2. 立方和公式：

$$1^3 + 2^3 + \dots + n^3 = \frac{n^4 + 2n^3 + n^2}{4}$$

3. 四次方和公式：

$$1^4 + 2^4 + \dots + n^4 = \frac{6n^5 + 15n^4 + 10n^3 - n}{30}$$

4. 五次方和公式：

$$1^5 + 2^5 + \dots + n^5 = \frac{2n^6 + 6n^5 + 5n^4 - n^2}{12}$$

5. 六次方和公式：

$$1^6 + 2^6 + \dots + n^6 = \frac{6n^7 + 21n^6 + 21n^5 - 7n^3 + n}{42}$$

6. 七次方和公式：

$$1^7 + 2^7 + \dots + n^7 = \frac{3n^8 + 12n^7 + 14n^6 - 7n^4 + 2n^2}{24}$$

7. 八次方和公式：

$$\begin{aligned} &1^8 + 2^8 + \dots + n^8 \\ &= \frac{10n^9 + 45n^8 + 60n^7 - 42n^5 + 20n^3 - 3n}{90} \end{aligned}$$

8. 九次方和公式：

$$\begin{aligned} &1^9 + 2^9 + \dots + n^9 \\ &= \frac{2n^{10} + 10n^9 + 15n^8 - 14n^6 + 10n^4 - 3n^2}{20} \end{aligned}$$

9. 十次方和公式：

$$\begin{aligned} &1^{10} + 2^{10} + \dots + n^{10} \\ &= \frac{6n^{11} + 33n^{10} + 55n^9 - 66n^7 + 66n^5 - 33n^3 + 5n}{66} \end{aligned}$$

10. 等比級數：

$$S = a \cdot \frac{r^n - 1}{r - 1}$$

11. 二項式係數恆等式：

$$\binom{n}{0} + \binom{n}{1} + \dots + \binom{n}{n} = 2^n$$

$$\binom{n}{1} + \binom{n}{2} + \dots + \binom{n}{n} = 2^n - 1$$

$$\binom{n}{1} + \binom{n}{3} + \dots + \binom{n}{n-k} = 2^{n-1}$$

12. 分配問題 (玩具分給小孩)：

(a) n 個玩具 · k 位小孩 · 可以有人沒拿到：

$$\binom{n+k-1}{n} = \binom{n+k-1}{k-1}$$

(b) n 個玩具 · k 位小孩 · 每個人至少一個：

$$\binom{n-1}{k-1}$$

14.1.9 位元運算

(a) 位元條件：

$$(x + k) \& (y + k) = 0$$

(b) 加法恆等式 (利用 XOR 與 AND)：

$$a + b = (a \oplus b) + 2 \cdot (a \& b)$$

(c) OR 與 AND 的關係：

$$a \mid b = a + b - (a \& b)$$

(d) 交換兩數：

$$a = a \oplus b, \quad b = a \oplus b, \quad a = a \oplus b$$

(e) 取得最低位的 1：

$$x \& (-x)$$

(f) 清除最低位的 1：

$$x \& (x - 1)$$

(a) LCM:

$$\text{lcm}(a, b) = \frac{a \times b}{\text{gcd}(a, b)}$$

(b) LCM (prime factorization):

$$\text{lcm}(a, b) = \prod_{p \in \text{primes}} p^{\max(e_a, e_b)}$$

(c) GCD (prime factorization):

$$\gcd(a, b) = \prod_{p \in \text{primes}} p^{\min(e_a, e_b)}$$

(d) Counting divisors:

If $n = \prod_{i=1}^k p_i^{e_i}$, cnt is $\prod_{i=1}^k (e_i + 1)$

14.1.11 數論公式

13. Bezout's identity :

$$ax + by = \gcd(a, b) \quad (\text{必定存在整數解 } x, y)$$

14. 模指數運算 (冪的冪):

$$a^{b^c} \equiv a^{\text{power_mod}(b,c, \text{MOD}-1)} \pmod{\text{MOD}}$$

Pythagorean theorem :

$$(n^2 + m^2, 2nm, n^2 - m^2), \quad n > m > 0$$

Wilson's theorem :

$$(p-1)! \pmod{p} = n-1 \quad (p \text{ 為質數})$$

Catalan number :

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

海龍公式：

$$\text{Area} = \sqrt{s(s-a)(s-b)(s-c)}, \quad s = \frac{a+b+c}{2}$$

兩圓相交弦長：

$$\text{Length} = 2 \cdot \sqrt{\frac{(s-a)(s-b)}{s}}, \quad s = \frac{r_1 + r_2 + d}{2}$$

三角形內切圓半徑：

$$r = \frac{2 \cdot \text{Area}}{a + b + c}$$

三角形外接圓半徑：

$$R = \frac{abc}{4 \cdot \text{Area}}$$

三角形重心：

$$G = \left(\frac{x_1 + x_2 + x_3}{3}, \frac{y_1 + y_2 + y_3}{3} \right)$$

三角形垂心：

$$H = \left(\frac{x_1 \tan A + x_2 \tan B + x_3 \tan C}{\tan A + \tan B + \tan C}, \frac{y_1 \tan A + y_2 \tan B + y_3 \tan C}{\tan A + \tan B + \tan C} \right)$$

三角形內心：

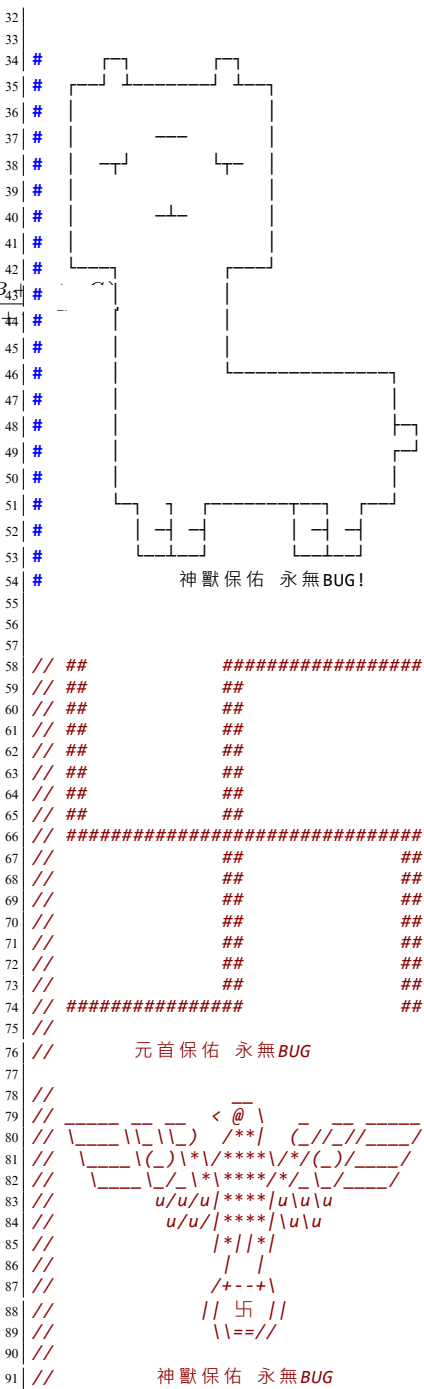
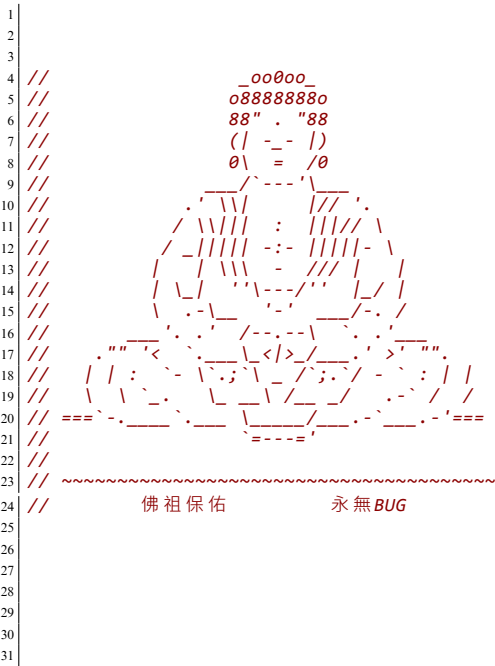
$$I = \left(\frac{ax_1 + bx_2 + cx_3}{a + b + c}, \frac{ay_1 + by_2 + cy_3}{a + b + c} \right)$$

奇 (偶) 長度陣列子數量：

$$n = \frac{(i + 1) * (n - i) + 1}{2}$$

15 Интернационал

15.1 保佑



ACM ICPC Team Reference - BogoSort

Contents

1 Algorithm	1	4.5 BIT - range update + range prefix sum	5	7.6 Sieve of Eratosthenes (with big num)	13	10.9 tree diameter	20
1.1 LIS	1	4.6 Order Statistics Tree	5	7.7 mod inv	13	10.10all longest path	20
1.2 LCS	1	4.7 Trie (Prefix tree)	5	7.8 derangement (DP)	13	10.11tree diameter (len,end)	20
2 Basic	1	4.8 segment tree	5	7.9 Euler Totient	13	10.12minimum distance in functional graph	20
2.1 mod helper function	1	4.9 BIT range update point query	5	7.10 fast power	13	10.13union length of two tree paths	20
2.2 self-defined-pq-operator	1	4.10 DSU	6	7.11 first and second mex	13	10.14Centroid decomposition	21
2.3 generating all subsets	1	5 Geometry	6	7.12 Catalan Number	13	11 default	22
2.4 tuple	1	5.1 cross. product	6	7.13 Chinese Remainder Theorem	13	11.1 debug	22
2.5 memset	1	5.2 Check Point in Polygon	6	7.14 mod inv (not prime)	13	11.2 template	22
2.6 submask enumeration	1	5.3 Point	6	7.15 mod inv (not coprime)	13	11.3 vimrc	22
2.7 stringstream split by comma	1	5.4 Polygon Lattice Points	7	7.16 inclusion-exclusion principle	13	12 other	22
3 DP	1	5.5 line intersect	7	7.17 C(n,k) mod inverse	14	12.1 Nim game	22
3.1 walk	1	5.6 Polygon area	7	7.18 Linear Diophantine 丢番圖	14	12.2 找小於 n 所有出現的 1 數量	22
3.2 grouping	1	5.7 using std::complex	7	7.19 擴展歐基里德	14	13 other language	22
3.3 matching	2	5.8 point distance from a line	8	7.20 Sieve of Eratosthenes	14	13.1 python heap	22
3.4 projects	2	6 Graph	8	7.21 builtin	14	13.2 java	22
3.5 elevator rides	2	6.1 Prim	8	7.22 C(n,k) DP	14	13.2.1 文件操作	22
3.6 digit sum	2	6.2 Eulerian cycle	8	8 Range Queries	14	13.2.2 优先队列	22
3.7 sushi	2	6.3 maximum weight bipartite matching	8	8.1 subarray maximum sum queries	14	13.2.3 Map	22
3.8 candies	3	6.4 maximum flow	9	8.2 find first equal or greater	15	13.2.4 sort	22
3.9 permutation	3	6.5 maximum cardinality bipartite matching	9	8.3 find k-th and count less than	15	13.3 python output	22
3.10 independent set	3	6.6 Floyd-Warshall	9	9 String	16	13.4 python 大數因數分解	22
3.11 counting tower	3	6.7 MST	10	9.1 Z	16	13.5 decimal	23
3.12 counting numbers	3	6.8 Hopcroft-Karp	10	9.2 manacher	16	13.6 python 大數排序	23
4 Data Structure	4	6.9 Dijkstra	10	9.3 AC 自動機	16	13.7 python 大數計算 2	23
4.1 undo disjoint set	4	6.10 Dinic	10	9.4 KMP	16	13.8 python input	23
4.2 lazy propagation	4	6.11 maximum cardinality matching	10	9.5 suffix array lcp	17	13.9 python 大數計算	23
4.3 Lazy heap	5	6.12 topological sort	11	9.6 hash	17	13.10base-6 pow	23
4.4 Fenwick tree (BIT)	5	6.13 Flyod's algorithm	11	9.7 minimal string rotation	17	14 zformula	23
		6.14 stable matching	12	9.8 reverseBWT	17	14.1 formula	23
		6.15 Bellman-Ford	12	10 Tree	17	14.1.1 Pick 公式	23
		7 Number Theory	12	10.1 Euler tour+RMQ	17	14.1.2 圖論	24
		7.1 Linear Sieve	12	10.2 HLD template	17	14.1.3 學長公式	24
		7.2 C(n,m)	12	10.3 all longest path dfs	18	14.1.4 基本數論	24
		7.3 Euler Totient 1 to n	12	10.4 all longest path top sort	18	14.1.5 排組公式	24
		7.4 derangement (Principle of Inclusion-Exclusion)	12	10.5 Heavy-light decomposition (HLD)	18	14.1.6 冪次, 冪次和	24
		7.5 matrix template (with fast power)	12	10.6 get kth ancestor	19	14.1.7 循環小數轉分數	24
				10.7 path maximum edge between u, v in tree	19	14.1.8 常見級數與組合公式	24
				10.8 binary jumping	19	14.1.9 位元運算	24
						14.1.10 除數與倍數	25
						14.1.11 數論公式	25

15	Интернационал	25	15.1	保佑	25
-----------	----------------------	-----------	------	--------------	----